

Securing Your MCP Server End-to-End:

IBM Verify + HashiCorp Vault

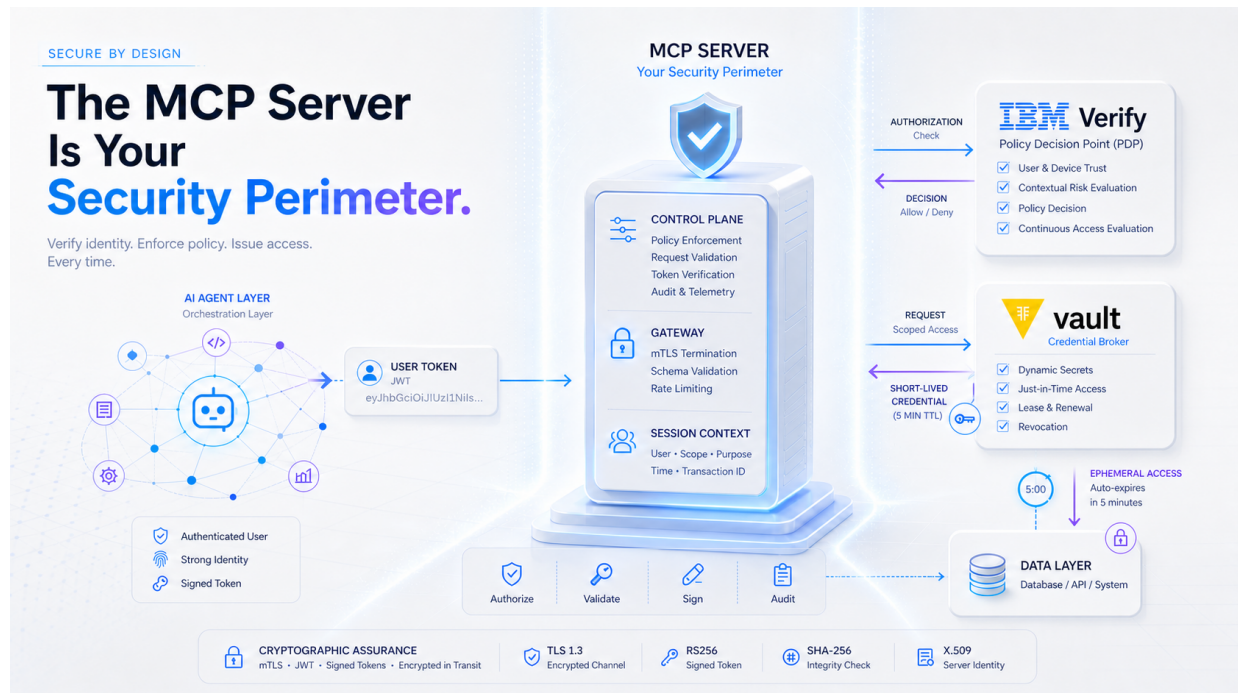
A 30-minute hands-on walkthrough that lands a working MCP server on your laptop where every tool call is authorized by IBM Verify policy (with step-up MFA where the policy demands it) and runs under a 5-minute PostgreSQL credential that HashiCorp Vault mints fresh per call. No AWS account required.

2026-05-22



Robert Graham
rgraham@us.ibm.com
Global Product Architect - Verify

MCP Server Reference Cookbook



The MCP server is the security perimeter. The agent forwards the user token; IBM Verify makes the policy decision; Vault mints the short-lived credential; the data layer only sees ephemeral access.

Table of Contents

1. Architecture
2. The identity chain
3. Prerequisites
4. Clone the repo
5. Configure HashiCorp Vault
6. Configure IBM Verify
7. Configure PostgreSQL
8. Start the MCP server
9. Run the agent
10. End-to-end smoke test
11. Swapping the LLM
12. Anatomy of an MCP call
13. Troubleshooting
14. Logging for an enterprise SIEM

1. Architecture

Deployment and Architecture

You are about to wire up a small but complete identity and authorization chain. Read this once, slowly. Spending five minutes here saves an hour of confusion later.

The picture

Diagram placeholder: Replace this page with the final customer architecture diagram if you want a different visual treatment. This draft uses mcp-header-light.png as the diagram replacement.

A clinician on a workstation signs into the agent with their IBM Verify token. The agent forwards that token to the MCP server. The MCP server exchanges the token at IBM Verify (RFC 8693 Token Exchange) and asks for a scoped on-behalf-of token, attaching a Rich Authorization Request (RFC 9396) that describes the exact action it wants to perform. IBM Verify evaluates a policy against the request; for high-risk reads it triggers a step-up MFA push on the clinician's phone. Once the policy allows the action, IBM Verify mints a short-lived OBO token. The MCP server presents that OBO to HashiCorp Vault, where the verify-rar plugin validates the JWT signature against the Verify JWKS, matches the embedded authorization details to a role mapping, and mints a 5-minute PostgreSQL credential. The MCP server uses that credential to run a single SELECT, returns the row, and the lease expires.

What you are building, in one paragraph

A working reference for an AI agent that does the right thing under enterprise identity. The clinician's identity flows end-to-end as standards-based JWTs. The agent itself never holds long-lived database credentials. Each tool call mints a fresh, scoped, time-bound credential and discards it. High-risk reads require a human to approve with their phone before any data leaves the database. Every step writes audit records that share a single correlation id (jti) so a SOC can replay any chain from the SIEM.

The seven steps, one paragraph each

Step 1: Clinician to Agent. A clinician opens a chat with the agent and submits a prompt such as "show me patient A0042's record." The clinician's browser or calling application carries the clinician's access token from IBM Verify in the request to the agent.

Step 2: Agent to MCP server. The agent decides which tool to invoke (here, `get_patient_record`). It opens a Model Context Protocol connection to the local MCP server and forwards the clinician's access token as a Bearer header. The agent does no authorization decisions; it is a transport.

Step 3: MCP server to IBM Verify. The MCP server takes the clinician's token as the `subject_token` for an RFC 8693 Token Exchange. It builds an RFC 9396 `authorization_details` block describing the action (operation = `read_patient_record`, location = `mrn:A0042`) and POSTs the exchange to IBM Verify. This is the only place in the system that uses the long-lived Token Exchange client secret.

Step 4: IBM Verify policy evaluation and step-up MFA. IBM Verify reads the `authorization_details` and evaluates the access policy bound to the Token Exchange application. For the VIP read, the policy fires

ACTION_MFA_ALWAYS: Verify returns a `mfa_challenge` token (not an OBO) on the first leg, the MCP server uses it to trigger a push at `/v1.0/authenticators/{factor_id}/verifications` and polls the transaction URI until the clinician approves, then submits a second-leg exchange with `grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer` re-sending the same `authorization_details`. Verify mints a short-lived OBO JWT carrying the original RAR and a unique `jti` claim. For non-VIP reads, the policy fires **ACTION_ALLOW** and the OBO comes back on the first leg with no push.

Step 5: MCP server to Vault. The MCP server presents the OBO JWT directly to HashiCorp Vault as the `x-Vault-Token` header. Vault's `verify-rar` plugin validates the JWT signature against the Verify JWKS, matches the embedded `authorization_details` to one of the role's `rar_mappings`, and mints a fresh PostgreSQL role with a 5-minute lease.

Step 6: MCP server to PostgreSQL. The MCP server opens a database connection as the ephemeral role (the role name and password came back from Vault) and runs a single `SELECT` against the `clinical.patients` table. The database logs the statement under the ephemeral role name.

Step 7: Lease revoke and return. The MCP server explicitly revokes the Vault lease as soon as the `SELECT` completes. The ephemeral PostgreSQL role is dropped. The MCP server returns the patient record to the agent, the agent streams the answer back to the clinician, and the chain ends.

What lives where

Four processes run on your machine. You start each one yourself and you can see all of their logs.

1. **PostgreSQL container.** The clinical database. Tables: `clinical.clinicians`, `clinical.patients`, `clinical.visits`. Seeded with ten patients (two flagged VIP) and one clinician. Comes up with `docker compose up -d postgres`.
2. **HashiCorp Vault container, in dev mode.** Hosts the `verify-rar` plugin that does the per-call credential mint. Comes up with `docker compose up -d vault`. Bootstrap creates the role, the policy, and a 24-hour token for the MCP server.
3. **MCP server, on your host.** A Node 22 process listening on `127.0.0.1:3012`. Handles Model Context Protocol requests, performs the Token Exchange, talks to Vault, queries PostgreSQL.
4. **Agent process, on your host.** A Python 3.11+ FastAPI server listening on `127.0.0.1:8080`. Runs the Strands agent loop. Receives `POST /invoke` from your calling app with the clinician's Bearer token, forwards the token to the MCP server, streams the model's response back.

No cloud accounts and no AWS dependencies. The agent has no AWS account requirement, no IAM role, no managed runtime. It is just a Python process.

What you are NOT installing

The local stack is intentionally small. The following are common assumptions about an enterprise agent platform that you do not need for this cookbook:

- No AWS account, no Bedrock subscription, no IAM policies. The agent runs locally as a Python process; the default LLM is Anthropic direct.
- No managed agent-hosting runtime. The agent is `uvicorn` running a FastAPI app, on your laptop or anywhere else that runs Python.
- No Kubernetes. The three local services run as plain Docker containers, a Node process, and a Python process.

- No Cloudflare or other tunnel service. Everything binds to 127.0.0.1.
- No NAT gateway, VPC endpoint, or private link. The local stack does not need outbound access beyond Verify and your chosen LLM provider.
- No SPIRE server. Workload identity to Vault is unnecessary because the OBO JWT itself is the credential Vault validates.
- No central event bus, CAEP transmitter, or session-revoke fabric. Auditing is via the standard surfaces (Vault audit, PostgreSQL log, agent log, Verify event stream) and is covered in docs/siem-logging.md.
- No bespoke aggregator or correlation engine. Every record carries the OBO jti and a SIEM joins on that field.

For this test deployment, they are not in the critical path for a working end-to-end demo on a laptop.

What you just did

You read the system end-to-end. You can name the actor on each side of every step and you know which of the four services owns which step.

What you'll do next

Move on to Configure IBM Verify to set up the tenant-side pieces: the custom attribute that detects the healthcare RAR, the access policy that triggers step-up MFA, and the two OIDC applications the rest of the chain refers back to.

2. The identity chain

The Identity Chain

This is the security story in one chapter. The MCP server in this cookbook is the only process that makes IBM Verify calls, the only one that handles the on-behalf-of token, the only one that talks to HashiCorp Vault. Everything that protects a patient record is concentrated in that one process. Read this chapter before the implementation chapters so the chain's *shape* is clear before you stand it up piece by piece.

The five-second version

> Clinician bearer in -> MCP server builds an RFC 9396 Rich Authorization Request -> IBM Verify policy decides (with step-up MFA where the policy demands it) -> Verify signs the RAR into an OBO JWT -> HashiCorp Vault validates the OBO and matches its RAR -> ephemeral 5-minute PostgreSQL role -> one SELECT -> lease revoked.

Each arrow is a standards-based wire format your existing tooling can already inspect: OAuth 2.0 / OIDC for the user sign-in, RFC 8693 for the Token Exchange, RFC 9396 for the authorization details, Vault audit JSON for the credential mint, PostgreSQL `log_statement=all` for the database call.

Three guarantees that live in IBM Verify, not in the code

These are the assurances a security reviewer is looking for. None of them live in the MCP server or the agent; all three are properties of how IBM Verify enforces the access policy.

1. Per-call authorization, not per-session. Every single tool invocation runs its own Token Exchange. The MCP server cannot accumulate or reuse authority across calls. A clinician who has read patient A0001 has *not* been pre-approved to read A0042; the next call goes through Verify again and gets a fresh evaluation.

2. Step-up MFA evaluated at policy time. When the cookbook's access policy fires `ACTION_MFA_ALWAYS` for VIP reads, IBM Verify returns `scope: mfa_challenge` instead of an OBO. The MCP server triggers a push at IBM Verify's `/v1.0/authenticators/{factor_id}/verifications` endpoint and polls until the clinician approves. Only then does Verify mint the real OBO. There is no path through the MCP server's code that bypasses this gate; the policy decision happens *inside Verify*, not inside the cookbook.

3. Policy-attested authorization details. The RAR the MCP server sends to Verify is *cryptographically attested* on the way back: Verify signs the approved `authorization_details` into the OBO JWT it returns. Vault then validates that JWT signature against the Verify JWKS *before* it matches the RAR to a `rar_mappings` entry. If the MCP server tried to forge a wider RAR locally and present it to Vault directly, Vault would reject the signature check.

What the MCP server is forbidden from doing

A security reviewer who picks up this code will want to know what authorization decisions the MCP server can make unilaterally. The answer is: almost none. The forbidden-list is short and worth reading in full.

- **No local allow/deny logic.** The MCP server never reads the clinician bearer's claims to decide if they can see a patient. It forwards the token to Verify as `subject_token` and acts on Verify's response.
- **No OBO caching across calls.** Every tool call mints a fresh OBO with a fresh `jti` claim. There is no opportunity for the MCP server to reuse an old OBO for a new request. Vault audit can correlate every PostgreSQL mint to exactly one IBM Verify policy decision.
- **No long-lived database credentials.** The MCP server never holds a PostgreSQL password. It uses the 5-minute ephemeral role HashiCorp Vault mints from the OBO, runs one SQL statement, and explicitly revokes the lease.
- **No hand-rolled crypto.** OBO JWT signature validation happens in HashiCorp Vault, against the IBM Verify JWKS. The MCP server does not introspect the OBO or implement any JWT verification of its own.

What the agent is forbidden from doing

If the MCP server is "almost nothing" from an authorization standpoint, the **agent** is literally nothing. The agent is a transport, not a security boundary.

- **The agent never inspects the clinician bearer.** It reads the `Authorization: Bearer` header off the inbound HTTP request, forwards it verbatim to the MCP server, and that is the end of the agent's relationship with the token.
- **The agent never talks to IBM Verify.** It has no Verify client credentials, no Verify tenant URL hard-coded, no `/oauth2/token` calls in its source. If you `grep` the agent code for the string `verify`, the only match is the URL the MCP server is hosted at.
- **The agent never talks to HashiCorp Vault.** It has no Vault token, no `/v1/verify-rar/creds/...` call, no policy access. The credentials in Vault never reach the agent process.
- **The agent never touches the database.** It has no PostgreSQL connection string, no PG library installed in its dependencies, no SQL.

The agent's job is to think and route. A change of agent framework, host, or even programming language does not change the security chain. The MCP server is the security perimeter; the agent is just a different way to call it.

The trust boundary, sliced one more way

When a reviewer asks "where is the trust boundary in this system?" the answer is the MCP server. Specifically, three load-bearing files inside `mcp-server/src/`:

5. The HTTP entrypoint that refuses any request without a Bearer header (`index.ts`).
6. The tool handler that constructs a fresh RFC 9396 RAR per call (`tools/get-patient-record.ts`).
7. The Token Exchange handler that implements the RFC 8693 + `mfa_challenge` two-leg flow (`verify/token-exchange.ts`).

The chapter Anatomy of an MCP Call opens each of those three files near the end of the cookbook (after you have stood up the stack and watched it work) and walks through the code line by line. Read it after the smoke-test chapter, not before; the code lands better when you have already seen it run.

What you just did

You read the security story end-to-end without yet running any code. You know which guarantees live in IBM Verify (per-call authz, step-up MFA, RAR attestation), what the MCP server is forbidden from doing on its own (local allow/deny, OBO caching, long-lived DB creds, hand-rolled JWT verification), and that the agent is a transport with no security responsibilities. You can quote any of those points back to a security reviewer without checking the code first.

What you'll do next

Move on to Prerequisites to confirm you have Docker, Node, Python, and an IBM Verify tenant ready, then on to the implementation chapters that stand up each piece of the chain.

3. Prerequisites

Screenshot placeholder: Optional prerequisites screenshot or checklist before starting the hands-on build.

Have the following installed and ready before you start the implementation chapters. The first run takes about 30 minutes; subsequent runs (with the bootstraps already done) are a few minutes.

- Docker (Docker Desktop on Mac, Docker Engine on Linux, 4.x or newer; Rancher Desktop running `dockerd` also works).
- Node.js 22.x and npm.
- Python 3.11 or newer.
- HashiCorp Vault CLI (optional; the bootstrap script uses the container's CLI by default).
- `jq` (the bootstrap scripts use it to parse JSON).
- `curl` (the Vault bootstrap script uses it to POST role definitions to the Vault API; standard on Mac and Linux).
- An IBM Verify tenant where you can create OIDC applications and access policies.
- The IBM Verify mobile app installed on a phone, with a userPresence (push) factor enrolled for the clinician test account. The VIP step-up smoke test in chapter 10 fires a push to that device; without an enrolled factor the VIP read returns `mfa_no_factor` (see troubleshooting #8). The non-VIP read still works without it.
- An LLM API key. The default in chapter 9 is Anthropic direct (`ANTHROPIC_API_KEY` from `console.anthropic.com`). If you want a different provider, see Swapping the LLM for what you need instead.
- Around 30 minutes for the first run.

No AWS account is required. The cookbook stack is fully local: Docker for PostgreSQL and Vault, a Node process for the MCP server, a Python process for the agent. Hosting the agent on a cloud platform is your customer's call and not part of this cookbook.

Validated local run order

The chapter order explains the components in a security-first way: Vault, Verify, PostgreSQL, MCP server, agent, then smoke test. The order you run the stack is slightly different. Use this run order when you want a clean, repeatable local demo.

8. Start the Docker stack first: PostgreSQL and Vault need to be running before either bootstrap can write state.
9. Confirm PostgreSQL is healthy and seeded. The local demo should show ten patients, with two VIP records, and the `vva_admin` plus `healthcare_read_template` roles.
10. Run the IBM Verify bootstrap. This creates or updates the tenant objects and writes the Token Exchange and UI client secrets into Vault KV.
11. Run the Vault bootstrap after the Verify bootstrap. This registers the `verify-rar` plugin, writes the database config and role mapping, writes the MCP policy, and prints a fresh 24-hour MCP Vault token.
12. Paste the latest printed Vault token into `mcp-server/.env` as `VAULT_TOKEN`, then restart the MCP server so it reads the new token.
13. Mint a fresh clinician access token with `scripts/get-clinician-token.sh`, source `.clinician-token.env`, and then run `scripts/smoke-test.sh`.

If you reset or restart Vault in dev mode, treat Vault state as gone: re-run the Verify bootstrap, re-run the Vault bootstrap, update `VAULT_TOKEN`, and restart the MCP server before testing again.

4. Clone the repo

Screenshot placeholder: Terminal showing the git clone completed and the repo directory layout.

```
git clone https://github.com/iamidentity-ai/mcp-verify-vault.git
cd mcp-verify-vault
```

Expected: a new directory `mcp-verify-vault/` with `infra/`, `mcp-server/`, `agent/`, `docs/`, and `scripts/` subdirectories.

The chapters that follow are the same prose you see in the per-chapter files under `docs/`. The two are kept in lockstep by `scripts/assemble-cookbook.sh`; edit the per-chapter file and re-run the script to refresh this assembled document.

5. Configure HashiCorp Vault

Vault Deployment

HashiCorp Vault is the last-mile credential broker. It hosts the **verify-rar plugin**, which does the per-call PostgreSQL credential mint. When the MCP server presents an IBM Verify on-behalf-of (OBO) token to Vault, the plugin validates the JWT signature against the Verify JWKS, matches the Rich Authorization Request (RAR) embedded in the token against the role's rar_mappings, and issues a brand-new PostgreSQL role with a 5-minute lease and SELECT on the clinical schema. The agent never holds a long-lived database credential.

Obtain the plugin binary

The verify-rar plugin is the load-bearing piece of this cookbook — it validates each OBO JWT against the Verify JWKS and mints the ephemeral PostgreSQL role. The plugin's source repository is in the process of being published; until that lands, request the latest build (or source archive for an audit) by emailing **support@iamidentity.ai**. State the OS and CPU architecture of the Vault container you intend to run it in (almost always linux/arm64 for Apple Silicon Macs running Docker Desktop, or linux/amd64 for Intel Macs / Linux developers).

Validated update: the local bootstrap can now auto-build the plugin if infra/vault/plugins/vault-plugin-secrets-verify-rar is missing. By default it looks for the companion source repository at ../verify-rar-vault-plugin relative to this cookbook repo; set VERIFY_RAR_PLUGIN_SOURCE=/path/to/verify-rar-vault-plugin if your checkout lives somewhere else. You still need Go installed locally.

Once you have the binary, copy it into the mounted plugins directory of this repository:

```
cp vault-plugin-secrets-verify-rar \
  <path-to>/mcp-verify-vault/infra/vault/plugins/
```

Expected: the file lands at infra/vault/plugins/vault-plugin-secrets-verify-rar and is executable (chmod +x if not). The bootstrap script in the next section computes the SHA-256 itself before registering the plugin with Vault, so you do not need to compute it by hand.

There is a separate companion plugin that rotates IBM Verify client secrets out of Vault. **You do not need it for this cookbook.** It is useful when many production services need their own rotating Verify client; here, a single 24-hour Vault token for the MCP server is sufficient.

Deploying the plugin to other Vault environments

The plugin binary has to match the OS and CPU architecture of the **Vault process itself**, not the developer's laptop. The cookbook's local Vault runs inside a Docker container, so the binary has to be a Linux binary for that container's architecture. If you are taking the plugin to a different Vault deployment, request the matching build from support@iamidentity.ai, or (once you have the plugin source archive) build for that target. Five common scenarios:

This auto-build path is not Mac-only. The important target is the Vault runtime. For the cookbook Docker stack, Vault always runs Linux inside the container, so the script detects the Vault container architecture with uname -m and builds linux/arm64 on Apple Silicon or linux/amd64 on Intel/AMD Docker. On Windows,

Linux, or macOS the same rule applies as long as Docker and Go are available. For a non-container Vault server, build the plugin for that Vault server OS and CPU and place it in that server's configured plugin directory.

Scenario	Where Vault runs	Build command
Cookbook on Apple Silicon Mac	Docker Desktop, Linux arm64 container	GOOS=linux GOARCH=arm64 make build
Cookbook on Intel Mac	Docker Desktop, Linux amd64 container	GOOS=linux GOARCH=amd64 make build
Linux developer with native Docker	Linux container matches host architecture	make build (no cross-compile needed)
Linux + k3s (Kubernetes)	Pod on a k3s node, Linux + node architecture	On a node of the matching arch: make build. Cross-compiling from a Mac: GOOS=linux GOARCH=amd64 make build (most k3s nodes are amd64).
Red Hat Enterprise Linux, Vault binary running natively	RHEL host directly, almost always amd64	On the RHEL host: make build. Cross-compiling from a Mac: GOOS=linux GOARCH=amd64 make build.

After building, the binary has to land somewhere Vault can read it. The exact path depends on the deployment shape:

Cookbook (Docker Desktop): copy the binary into `infra/vault/plugins/`. The dev container has this directory mounted at `/vault/plugins/` (read-only), and the bootstrap script registers the binary by SHA. Nothing else to do.

Linux + k3s: the cleanest path is to bake the binary into a custom Vault image. Write a small Dockerfile FROM hashicorp/vault:1.18 ... COPY vault-plugin-secrets-verify-rar /vault/plugins/, push to your registry, and point the official Vault Helm chart at it via `injector.image` overrides. ConfigMap-plus-init-container works too but the plugin binary is around 30 MB which is close to ConfigMap's 1 MB limit, so a custom image is the maintainable choice. PVC plus manual `kubectl cp` works for one-time evaluation but is fragile for ongoing deployments.

RHEL non-container: copy the binary to `/etc/vault.d/plugins/vault-plugin-secrets-verify-rar` (or whatever `plugin_directory` your `vault.hcl` declares), `chown vault:vault, chmod 0750`. If SELinux is enforcing on the host, `chcon -t bin_t /etc/vault.d/plugins/vault-plugin-secrets-verify-rar` or write a small policy module. Then register in the catalog and enable the secrets engine the same way the bootstrap script does:

```
SHA=$(sha256sum /etc/vault.d/plugins/vault-plugin-secrets-verify-rar | awk '{print $1}')
vault plugin register -sha256=${SHA} secret vault-plugin-secrets-verify-rar
vault secrets enable -path=verify-rar -plugin-name=vault-plugin-secrets-verify-rar plugin
```

No Vault restart is needed for plugin registration. You only restart if you changed `plugin_directory` in `vault.hcl`.

HCP Vault Dedicated (HashiCorp's SaaS): customer-uploaded plugins are not supported. HashiCorp manages the binary surface of HCP Vault; you cannot drop in `vault-plugin-secrets-verify-rar` and there is no upload mechanism. If your customer is committed to HCP Dedicated, you have three options.

14. **Run a small self-managed Vault alongside HCP.** HCP holds whatever the customer originally chose it for; a separate self-managed Vault (Enterprise or OSS) hosts the `verify-rar` plugin. The MCP server is configured with two Vault addresses, one for static service secrets and one for the per-call mint. Workable but architecturally noisy.

15. **Move the verify-rar logic into the MCP server itself.** The plugin's job is to validate the OBO claims against rar_mappings and mint a Postgres role. None of that strictly requires Vault. The MCP server can perform the same check directly against Postgres, with HCP Vault holding only the static service secrets. You lose the Vault audit trail for per-call mints, but you keep the per-call RAR check.
16. **Stay self-managed for Vault, use HCP for non-Vault services only.** If the customer's HCP investment is in other HashiCorp services (HCP Boundary, HCP Consul, HCP Packer), this is often the simplest answer.

If HashiCorp opens a customer-plugin-upload feature on HCP Dedicated later (they have hinted at it but have not committed to a date), revisit option 1 and you can collapse to a single Vault deployment.

Bring up Vault

```
irgraham@mac verify-vault-agentcore-reference % curl -s http://127.0.0.1:8200/v1/sys/health | jq
{
  "initialized": true,
  "sealed": false,
  "standby": false,
  "performance_standby": false,
  "replication_performance_mode": "disabled",
  "replication_dr_mode": "disabled",
  "server_time_utc": 1779155158,
  "version": "1.18.5",
  "enterprise": false,
  "cluster_name": "vault-cluster-67adfddd",
  "cluster_id": "385f25bf-3434-9744-d363-dfddf8fab1f4",
  "echo_duration_ms": 0,
  "clock_skew_ms": 0,
  "replication_primary_canary_age_ms": 0
}
```

Vault health check: initialized=true, sealed=false.

The local stack runs Vault in dev mode inside a container. Dev mode auto-unseals on every start and has its own root token printed in the logs; this is intentional for a local-only walkthrough. Do not point production traffic at a dev-mode Vault.

```
cd infra
docker compose up -d vault
```

Expected: Container vva-vault Started.

Wait a few seconds for Vault to initialize, then confirm it is responding:

```
curl -s http://127.0.0.1:8200/v1/sys/health | jq
```

Expected (abridged):

```
{
  "initialized": true,
  "sealed": false,
  "standby": false
}
```

If sealed is true, your container did not start in dev mode. Check `docker logs vva-vault` and confirm you see Root Token: near the top.

Configure infra/.env before bootstrapping

The bootstrap script reads its inputs from `infra/.env`. The Docker compose stack reads the same file for the Vault dev token and the Postgres admin credentials, so this one file feeds both. Copy the example and fill in your Verify tenant hostname:

```
cp infra/.env.example infra/.env
```

Open `infra/.env` and set the one value that has no safe default:

```
VERIFY_TENANT_HOST=<your-tenant>.verify.ibm.com
```

The remaining values match the local Vault container and Postgres container the cookbook brings up; they are pre-filled and you do not need to change them unless you are pointing at a different stack:

```
POSTGRES_ADMIN_USER=vva_admin
POSTGRES_ADMIN_PASSWORD=vva_admin_local_dev_only
POSTGRES_DB=healthcare
VAULT_ADDR=http://127.0.0.1:8200
VAULT_DEV_ROOT_TOKEN=vva-dev-root-token
```

If you set `VERIFY_TENANT_HOST` and run the bootstrap, the script synthesizes the full URL (`https://${VERIFY_TENANT_HOST}`) and the JWKS endpoint (`${VERIFY_TENANT_HOST}/oauth2/jwks`) automatically.

Run the bootstrap

```
rgraham@mac verify-vault-agentcore-reference % bash infra/vault/bootstrap-vault.sh
[bootstrap] Vault addr: http://127.0.0.1:8200
[bootstrap] Plugin: vault-plugin-secrets-verify-rar
[bootstrap] Role: healthcare-records

[bootstrap] Plugin SHA256: 216f0279dfd47f0cba212d8631828bb97b6f7fc04ee5f386e15df37b9e8e7c18
Success! Registered plugin: vault-plugin-secrets-verify-rar
[bootstrap] Plugin registered in catalog.
[bootstrap] Secrets engine already enabled at verify-rar/.
Success! Data written to: verify-rar/config/db
[bootstrap] Database connection written to verify-rar/config/db/healthcare.
[bootstrap] JWT claims will be passed in the request body by the MCP server (Vault OSS path).
[bootstrap] Role healthcare-records written with 5-minute TTL.
[bootstrap] KV v2 secrets engine already enabled at secret/.
Successfully copied 2.56kB to vva-vault:/tmp/healthcare-mcp.hcl
Success! Uploaded policy: healthcare-mcp
[bootstrap] Policy healthcare-mcp written.

=====
MCP server Vault token (24h TTL, policy healthcare-mcp):
hvs.CAESIJfuA-DSUgJEekyS23NeCLBz25s8FoUJfyYQta_wSR7zGh4KHGh2cy55bHEzUXZJVEgxcVhnRFZ4NH04eXBP0E

Put this in mcp-server/.env as VAULT_TOKEN=...
=====
```

Vault bootstrap output with the MCP Vault token.

The bootstrap registers the plugin in Vault's catalog, enables the secrets engine at `verify-rar/`, writes the database connection config, writes the role and its `rar_mappings`, writes the access policy for the MCP server, and mints a 24-hour token for the MCP server to use.

Run this after PostgreSQL is healthy and after the Verify bootstrap has written the Verify client secrets to Vault KV. In a clean local setup, the command should print the plugin SHA, register the plugin, write verify-

rar/config/db, write the healthcare-records role, upload the healthcare-mcp policy, and print the MCP server Vault token.

```
bash infra/vault/bootstrap-vault.sh
```

Expected output (abridged):

```
[vault-bootstrap] Plugin registered: vault-plugin-secrets-verify-rar
[vault-bootstrap] Secrets engine enabled at verify-rar/
[vault-bootstrap] Database connection healthcare-postgres: WRITTEN
[vault-bootstrap] Role healthcare-records: WRITTEN
[vault-bootstrap] Policy healthcare-mcp: WRITTEN
```

```
=====
MCP-server Vault token (paste into mcp-server/.env as VAULT_TOKEN):
  hvs.AAAA...
TTL: 24h
=====
```

Copy the printed `hvs...` token. You will paste it into `mcp-server/.env` as `VAULT_TOKEN` in chapter 7 (Start the MCP server). The token is renewable, so 24 hours is a forgiving window for a local development session.

Token freshness matters. The printed Vault token is the one the MCP server must use for this run. If you re-run the Vault bootstrap, copy the new token into `mcp-server/.env` and restart the MCP server. If Vault dev mode was restarted or rebuilt, old Vault tokens and old KV secrets should be assumed invalid.

What the bootstrap just did

Five Vault objects exist now. Read the list once so the `VAULT_ROLE=healthcare-records` value the MCP server uses in chapter 7 is unsurprising.

17. **The plugin in the catalog.** Vault knows about `vault-plugin-secrets-verify-rar` and the binary it can execute.
18. **The secrets engine mounted at ``verify-rar/``.** This is the path the MCP server POSTs to: `POST /v1/verify-rar/creds/healthcare-records`.
19. **The database connection config.** Vault holds the credentials for an admin PostgreSQL role (`vva_admin`) that has the right to `CREATE ROLE`, `GRANT`, and `DROP ROLE` against the `clinical` schema.
20. **The role ``healthcare-records``.** The role definition includes the SQL template that mints a new ephemeral role with `SELECT` on `clinical.patients`, `clinical.visits`, and `clinical.clinicians`, plus the `rar_mappings` that say which RAR shapes are allowed to use this role.
21. **The KV v2 secrets engine at ``secret/``.** Vault dev mode mounts this by default; the bootstrap re-asserts it so the same script works against a non-dev Vault. The Verify bootstrap in the next chapter writes the Token Exchange `client_secret` here at `secret/data/VERIFY_TE_CLIENT_SECRET`; if you ran the chapters in order, that value is already in Vault.
22. **The policy ``healthcare-mcp``.** This is the Vault policy attached to the 24-hour token. It grants update on `verify-rar/creds/healthcare-records`, update on `sys/leases/revoke`, and read on `secret/data/VERIFY_TE_CLIENT_SECRET` so the MCP server can fetch its own service secret from Vault rather than reading it from `.env`.

Common gotcha: the policy needs update, not read

The verify-rar plugin path `verify-rar/creds/<role>` is registered for both Vault's `ReadOperation` and `UpdateOperation`. Vault's HTTP layer does not deserialize a JSON request body on a GET, so the plugin needs a POST. POST corresponds to update in Vault's capability model.

If you write a policy that grants only read on this path, your CLI test with `vault read` will work fine (it goes through the read path), but the workload will get a silent HTTP 403 because it POSTs (which goes through the update path). The included `healthcare-mcp.hcl` policy grants **both** capabilities; you do not need to do anything. This note is for any future contributor who writes a new policy and stares at a 403 for an hour.

What you just did

You obtained the verify-rar plugin binary, dropped the binary into the mounted plugins directory, brought up a local dev-mode Vault, and ran the bootstrap that created the role, the policy, and a 24-hour token for the MCP server. You wrote down the `hvs.token`; chapter 7 (Start the MCP server) wants it.

What you'll do next

Move on to Configure IBM Verify to create the tenant-side attribute, access policy, and OIDC applications, then bring up PostgreSQL and the MCP server.

6. Configure IBM Verify

Verify Deployment

IBM Verify is the policy decision point for every action the agent takes. Before any of the other pieces work you need three things on your tenant: a **custom attribute** that recognizes the healthcare Rich Authorization Request (RAR), an **access policy** that triggers step-up MFA for VIP reads, and the **two OIDC applications** the agent and the MCP server use (a UI app for clinician sign-in, a Token Exchange app for the on-behalf-of flow). A small bootstrap script in `infra/verify/` creates all of this for you once you give it admin credentials.

Pre-requisite: create the admin API client

AgentCore source screenshots reused for the shared IBM Verify API-client workflow. Retake this sequence if you want the final images to show the MCP-specific project name.

-  **Home**
-  **Applications** 
-  **Directory** 
-  **Authentication** 
-  **Security** 

Access policies

Portal access

Endpoint settings

OIDC tokens

API access

Certificates

Application consents

Password management

Sign-in options

Threat detection

The bootstrap script calls the IBM Verify admin API. That API needs its own API client with the right entitlements. Create it once in the Admin UI; the rest of this chapter is scripted.

23. Sign in to your IBM Verify Admin UI at <https://<your-tenant>.verify.ibm.com>.

24. Open **Security** -> **API**, click **Create API client**, and name it something like `mcp-verify-vault-admin`.

25. Under **Entitlements**, grant all five of the following. The bootstrap script touches custom attributes, access policies, OIDC apps, and per-app entitlement assignment, and each of those operations needs its own entitlement.

- Manage access policies
- Manage application entitlements
- Manage application lifecycle
- Manage attribute sources
- Manage entitlements

26. Save the API client. On the resulting page, copy the **Client ID** and **Client Secret**.

Keep both values handy. You will write them to Vault in the next step.

Land the admin credentials in Vault

```
rgraham@mac infra % docker exec -e VAULT_TOKEN=vva-dev-root-token vva-vault \
  vault kv put secret/VERIFY_ADMIN_CREDS \
    client_id=5d6931c1-7cbd-4666-b58d-66e0e21673a2 \
    client_secret=9tVm2OVDWD
===== Secret Path =====
secret/data/VERIFY_ADMIN_CREDS

===== Metadata =====
Key                               Value
----                               -
created_time                      2026-05-19T01:49:34.126113785Z
custom_metadata                   <nil>
deletion_time                     n/a
destroyed                         false
version                           1
```

Vault KV write for VERIFY_ADMIN_CREDS.

This is a privileged credential. It can modify any application, policy, or custom attribute on your tenant. Vault is already running from the previous chapter, so the credential goes straight there. It never sits in a file on disk.

```
docker exec -e VAULT_TOKEN=vva-dev-root-token vva-vault \
  vault kv put secret/VERIFY_ADMIN_CREDS \
    client_id=<the client id you just copied> \
    client_secret=<the client secret you just copied>
```

Expected:

```
=== Secret Path ===
secret/data/VERIFY_ADMIN_CREDS

===== Metadata =====
Key                               Value
```

```
---          -----
created_time    <timestamp>
custom_metadata <nil>
deletion_time   n/a
destroyed       false
version         1
```

The bootstrap script reads `client_id` and `client_secret` from this path on every run. If you ever rotate the admin credential in the Verify Admin UI, re-run the `vault kv put` command above; the next bootstrap run picks up the new value.

Configure your .env

Move into `infra/verify/` and copy the example file:

```
cd infra/verify
cp .env.example .env
```

Open `.env` in your editor. Only one value needs your input:

```
VERIFY_TENANT_HOST=<your-tenant>.verify.ibm.com
```

The tenant host is just the hostname, not a URL; do not include `https://` and do not include a trailing slash.

The remaining variables are pre-filled with local-dev defaults that match the Vault container you brought up in the previous chapter:

```
VAULT_ADDR=http://127.0.0.1:8200
VAULT_TOKEN=vva-dev-root-token
```

When both are set, the bootstrap script writes the Token Exchange app's `client_secret` to Vault KV at `secret/data/VERIFY_TE_CLIENT_SECRET`. The MCP server reads it from there at runtime, so it never has to live in `mcp-server/.env`. If you blank these out, the bootstrap prints the secret to your terminal for manual paste; that fallback exists for any customer who wants to adopt this pattern without standing up Vault first.

This is why Verify bootstrap comes before the final Vault/MCP smoke run. The MCP server reads `VERIFY_TE_CLIENT_SECRET` from Vault on token exchange. If Vault was reset after Verify bootstrap ran, the KV secret is gone and token exchange will fail until you run the Verify bootstrap again.

Run the probe

```
rgraham@mac verify % npm install

up to date, audited 9 packages in 549ms

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
rgraham@mac verify % npm run probe

> vva-verify-bootstrap@0.1.0 probe
> tsx probe-verify.ts

Verify reachable: OK
  Tenant: myco-consulting.tryverify.ibm.com
  Admin client id: 5d6931c1... (from vault)
```

Verify probe success output.

A small read-only check confirms your tenant is reachable and your admin credentials work. Run it first so you do not waste time debugging the bootstrap with bad creds.

```
npm install
npm run probe
```

Expected:

```
Verify reachable: OK
  Tenant: <your-tenant>.verify.ibm.com
  Admin client id: a1b2c3d4... (from vault)
```

The first line is the success signal. The two indented lines echo back the tenant host, the first eight characters of your admin client id, and where the script loaded the credential from. `from vault` confirms that the `vault kv put` step landed cleanly; `from env` means the script fell back to reading `VERIFY_ADMIN_CLIENT_*` out of `.env`.

If the probe fails, three things to check, in order:

27. Did you copy the client secret correctly? Open the admin application in the UI, regenerate the secret if you are not sure, and put the new value into `.env`.
28. Is your tenant host correct? Run `curl -s https://<your-tenant>.verify.ibm.com/oauth2/.well-known/openid-configuration | head` and confirm you see a JSON document.
29. Does the admin client actually have the five entitlements (Manage access policies, Manage application entitlements, Manage application lifecycle, Manage attribute sources, Manage entitlements)? Open the app in the UI, scroll to **Entitlements**, verify the list.

Run the bootstrap

```
rgraham@mac verify % npm run bootstrap

> vva-verify-bootstrap@0.1.0 bootstrap
> tsx bootstrap-verify.ts

[verify] admin credentials loaded from vault
[verify] attribute healthcareRarPresent updated (id ff65fb0c-6c8a-4db1-a886-c3cbaf79967d)
[verify] policy HealthcareStepUp updated (id 1301380)
[verify] app vva-healthcare-ui created (id 4609384217133237895, clientId 9edf2013-da8b-4317-be11-a80c65c931e8)
[verify] app vva-healthcare-ui entitlements set to automatic access for all users
[verify] app vva-healthcare-token-exchange created (id 4609394530472911781, clientId 08038f1b-3c94-452c-953a-dfac8855e1f7)
[verify] app vva-healthcare-token-exchange entitlements set to automatic access for all users
[verify] wrote TE client secret to Vault at secret/data/VERIFY_TE_CLIENT_SECRET

-----

Verify bootstrap complete. Wrote verify-output.json

Copy these into your other .env files:
VERIFY_TENANT_HOST=myco-consulting.tryverify.ibm.com
VERIFY_UI_CLIENT_ID=9edf2013-da8b-4317-be11-a80c65c931e8
VERIFY_TE_CLIENT_ID=08038f1b-3c94-452c-953a-dfac8855e1f7

VERIFY_TE_CLIENT_SECRET is in Vault at secret/data/VERIFY_TE_CLIENT_SECRET.
The MCP server reads it from Vault, so DO NOT put it in mcp-server/.env.

IMPORTANT: open the access policy in the Verify Admin UI, click
Save then Publish so it goes ACTIVE. API-created policies sit in
IDLE state until you do this once.
-----
```

Verify bootstrap output.

The bootstrap is idempotent. It checks for each object before creating it; running twice does not duplicate anything.

```
npm run bootstrap
```

Expected output (abridged):

```
[verify] attribute healthcareRarPresent created (id <attr-id>)
[verify] policy HealthcareStepUp created (id <policy-id>)
[verify] NOTE: API-created policies start in IDLE state. Open the policy in the Verify Admin UI, click
Save, then Publish, so it transitions to ACTIVE before binding it to the TE app.
[verify] app vva-healthcare-ui created (id <app-id>, clientId <ui-client-id>)
[verify] app vva-healthcare-token-exchange created (id <app-id>, clientId <te-client-id>)
[verify] wrote TE client secret to Vault at secret/data/VERIFY_TE_CLIENT_SECRET
```

```
-----
Verify bootstrap complete. Wrote verify-output.json
```

```
Copy these into your other .env files:
VERIFY_TENANT_HOST=<your-tenant>.verify.ibm.com
VERIFY_UI_CLIENT_ID=<ui-client-id>
VERIFY_TE_CLIENT_ID=<te-client-id>
```

```
VERIFY_TE_CLIENT_SECRET is in Vault at secret/data/VERIFY_TE_CLIENT_SECRET.
The MCP server reads it from Vault, so DO NOT put it in mcp-server/.env.
```

```
IMPORTANT: open the access policy in the Verify Admin UI, click
Save then Publish so it goes ACTIVE. API-created policies sit in
IDLE state until you do this once.
-----
```

Three values to copy into mcp-server/.env later: VERIFY_TENANT_HOST, VERIFY_UI_CLIENT_ID, VERIFY_TE_CLIENT_ID. **The Token Exchange client secret is in Vault now**, not in any .env file. The MCP server fetches it from secret/data/VERIFY_TE_CLIENT_SECRET on its first Token Exchange call and caches it

in process for five minutes. This is the design promise of the cookbook: the MCP server's `.env` holds one static credential (its Vault token), and Vault holds everything else.

If an object already exists from a prior run, the script prints updated instead of created and leaves the existing object in place. That is the intended idempotency: you can re-run the bootstrap to recover from a partial failure without duplicating anything.

The script also writes a `verify-output.json` file in the same directory; keep it around because the smoke-test chapter reads `VERIFY_UI_CLIENT_ID` out of it to drive the PKCE token grab.

If your environment has no Vault

The bootstrap detects this by reading `VAULT_ADDR + VAULT_TOKEN` from this directory's `.env`. When both are blank, the script skips the Vault write and prints the TE secret in the copy-paste block instead:

```
Copy these into your other .env files:
VERIFY_TENANT_HOST=<your-tenant>.verify.ibm.com
VERIFY_UI_CLIENT_ID=<ui-client-id>
VERIFY_TE_CLIENT_ID=<te-client-id>
VERIFY_TE_CLIENT_SECRET=<te-client-secret>

(No Vault KV write happened. Set VAULT_ADDR + VAULT_TOKEN in this
directory's .env to land the secret in Vault automatically.)
```

In that case you paste all four values into `mcp-server/.env`. The MCP server's secret loader falls back to the `env` var when Vault is not reachable, so the same code runs against either pattern. This fallback exists for any customer adopting the pattern in a stack without Vault; the cookbook's local stack always has Vault, so you will not normally see this.

Publish the access policy in the UI

Visual reference only: the source screenshot uses the older AgentCore policy naming/state. Retake this for the MCP cookbook so it shows *HealthcareStepUp* in Active state.

The screenshot displays the 'Access policy editor' for a policy named 'HealthcareStepUp (Draft)'. The policy type is 'undefined'. The 'Draft' status is indicated by a green pill. The editor is divided into several sections: 'Basic settings' (Policy name: HealthcareStepUp, Policy description: Fire step-up MFA when the request carries a healthcare RAR), 'Policy rules' (Adaptive access: Disabled, Reauthentication: Disabled, External integrations: No providers were selected), and 'Details' (ID: 1301380, Created on: May 18, 2026, Created by: -, Last modified by: verify-vault-agentcore-admin (API client) 5d6931c1-7cbd-4666-b58d-66e0e21673a2, Last modified: May 18, 2026, Version: 0). A 'Publish' button is located in the top right corner.

Access policy editor from the AgentCore cookbook.

This is the step that catches first-time users. **Access policies created via the IBM Verify admin API land in state `'IDLE'`. They do not evaluate.** You have to open the policy in the Admin UI and click Save and Publish to move it to ACTIVE. Until you do, the Token Exchange will allow the VIP read with no step-up MFA, which is the opposite of what you want.

30. In the Admin UI, open **Security** then **Policies**.
31. Find **HealthcareStepUp** in the list.
32. Click it to open the policy editor.
33. Without changing anything, click **Save** at the bottom right.
34. Click **Publish**.
35. Refresh the policy list. The state column should now read **ACTIVE**.

Verify the change in the Admin UI: the **State** column for HealthcareStepUp should now read **ACTIVE**. (The probe script does a tenant reachability check only and does not inspect policy state, so it is not the right tool for this verification.)

Rollback

If you want to undo the bootstrap (for instance, you want a clean tenant before re-running with a different name), the rollback script removes everything the bootstrap created.

```
npm run rollback
```

Expected:

```
[rollback] OIDC app vva-healthcare-token-exchange: DELETED  
[rollback] OIDC app vva-healthcare-ui: DELETED  
[rollback] Access policy HealthcareStepUp: DELETED  
[rollback] Custom attribute healthcareRarPresent: DELETED
```

Rollback does not delete your admin API Access application; that one you created by hand in the UI and you keep using it.

What you just did

You created one admin client by hand, ran a probe that confirmed your credentials work, ran a bootstrap that created the custom attribute, the access policy, and the two OIDC applications, and you published the policy so it actually evaluates. You have a `verify-output.json` file holding the UI and Token Exchange client ids for later chapters, and the Token Exchange client secret is in Vault KV at `secret/data/VERIFY_TE_CLIENT_SECRET` (the MCP server fetches it from there at runtime).

What you'll do next

Move on to Configure PostgreSQL. Vault and Verify are now ready; the next step is the clinical database the ephemeral roles will read from.

7. Configure PostgreSQL

PostgreSQL Deployment

The clinical database is the simplest piece of the stack but it is also where the real protection shows. The schema has three tables in a `clinical` schema (clinicians, patients, visits) and a role template called `healthcare_read_template` that has `SELECT` on the `clinical` schema and nothing else. Every time the agent does a tool call, the `verify-rar` plugin creates a fresh ephemeral role that inherits from that template, the MCP server uses it for a single `SELECT`, and the role is dropped 5 minutes later. Bring this up first; the rest of the stack assumes it is there.

Bring it up

```
rgraham@mac verify-vault-agentcore-reference % cd infra
rgraham@mac infra % docker compose up -d postgres
[+] up 12/14
[+] up 16/16 tgres:16-alpine [#####] 104.9MB / 107.3MB Pulling
✓ Image postgres:16-alpine Pulled
✓ add514cbd82 Pull complete
✓ 31b46f45ead5 Pull complete
✓ 898e671b81b8 Pull complete
✓ e3f744540ed7 Pull complete
✓ 80c4a7d433e0 Pull complete
✓ 035027b1ba33 Pull complete
✓ f73a68d77011 Pull complete
✓ d17f077ada11 Pull complete
✓ 19c6d932355c Pull complete
✓ 58ee90d55eb9 Pull complete
✓ b98b53b194cb Pull complete
✓ 2b6bbb6a8a33 Download complete
✓ 01eab40d03a6 Download complete
✓ Volume infra_postgres-data Created
✓ Container vva-postgres Created
rgraham@mac infra % docker inspect --format='{{.State.Health.Status}}' vva-postgres
healthy
```

PostgreSQL container started and healthy.

The PostgreSQL container is configured by `infra/docker-compose.yml`. The schema and seed SQL files in `infra/postgres/` are mounted into `/docker-entrypoint-initdb.d`, which the official PostgreSQL image runs on the very first container start.

Validated local update: the Docker Postgres container is published on `127.0.0.1:15432` on the host to avoid colliding with a native Postgres already listening on `5432`. Inside Docker, Vault still reaches the database as `postgres:5432`. From the host, the MCP server should use `POSTGRES_HOST=127.0.0.1` and `POSTGRES_PORT=15432`.

If a customer wants to keep host port `5432`, first check for conflicts with `lsof -nP -iTCP:5432 -sTCP:LISTEN`. A silent port collision can make the MCP server connect to the wrong Postgres instance; the symptom is usually that Vault successfully mints an ephemeral role, but the MCP server then gets role does not exist when it connects to the database.

```
cd infra
docker compose up -d postgres
```

Expected: Container `vva-postgres` Started.

Wait a few seconds, then confirm the container is healthy:

```
docker inspect --format='{{.State.Health.Status}}' vva-postgres
```

Expected: healthy.

If the status sits at starting for more than ten seconds, check `docker logs vva-postgres`. The first start runs the schema and seed scripts inline, which adds a few seconds.

Verify the data

```
graham@mac infra % docker inspect --format='{{.State.Health.Status}}' vva-postgres
healthy
graham@mac infra % docker exec vva-postgres psql -U vva_admin -d healthcare \
-c "SELECT mrn, display_name, vip_flag FROM clinical.patients ORDER BY mrn;"
```

mrn	display_name	vip_flag
A0001	Alex Anderson	f
A0002	Beth Bell	f
A0003	Carlos Chen	f
A0004	Dana Davis	f
A0005	Evan Evans	f
A0006	Fiona Fischer	f
A0007	Gabe Garcia	f
A0008	Hana Hassan	f
A0042	Senator Riley Reed	t
A0099	CEO Taylor Thornton	t

(10 rows)

Seeded patient table with VIP flags.

Confirm the seed loaded by counting patients and checking the VIP flag:

```
docker exec vva-postgres psql -U vva_admin -d healthcare \
-c "SELECT mrn, display_name, vip_flag FROM clinical.patients ORDER BY mrn;"
```

Expected: 10 rows. Two of them have `vip_flag = t` (true). The non-VIP MRNs return a patient record without any step-up MFA; the two VIP MRNs trigger the IBM Verify policy and a push to the clinician's phone. You will exercise both paths in the smoke test chapter.

Confirm the role template exists:

```
docker exec vva-postgres psql -U vva_admin -d healthcare \
-c "\du healthcare_read_template"
```

Expected: a line showing `healthcare_read_template` with the attribute `Cannot login`. That last bit is intentional. The template is not for direct login. The `verify-rar` plugin uses it as the inheritance parent when it mints ephemeral roles; the ephemeral roles themselves have a password and `LOGIN`.

Confirm the admin role the Vault plugin uses:

```
docker exec vva-postgres psql -U vva_admin -d healthcare \
-c "\du vva_admin"
```

Expected: a line showing `vva_admin` with `Superuser` or at least `Create role` in the `attributes` column. The plugin needs the right to `CREATE ROLE`, `GRANT` membership, and `DROP ROLE`; the simplest local-dev setup is to make it a superuser.

What you might add

For most demos the seeded ten patients are fine. If you want to expand the dataset (different specialties, additional VIPs, longer visit histories), here are the patterns.

Add more patients. Open `infra/postgres/02_seed.sql`, copy one of the existing `INSERT INTO clinical.patients` rows, change the `mrn` and the other fields, paste at the bottom. The MRN values in the seed file follow the `A####` convention; pick something that does not collide.

Add more clinicians. Same file, the `INSERT INTO clinical.clinicians` section. The clinician's `upn` is what the agent passes when it requests the patient list; change `CLINICIAN_UPN` in `agent/.env` if you want the agent to act as a different clinician.

Add more visits. Same file again, the `INSERT INTO clinical.visits` section. Foreign-keyed to `patients.mrn`; make sure the patient exists.

Caveat: changes to the seed files do not re-run on an existing container. The PostgreSQL image only runs `/docker-entrypoint-initdb.d` scripts on the very first start of an empty data volume. You have two options:

36. **Destructive: rebuild from scratch.** `docker compose down -v postgres && docker compose up -d postgres`. The `-v` flag drops the data volume, so the next start sees an empty data directory and runs your edited seed.
37. **Non-destructive: apply your INSERTs directly.** `docker exec -i vva-postgres psql -U vva_admin -d healthcare < your-new-rows.sql`. This is the right call when you want to preserve any data your testing has already created.

What you just did

You started a local PostgreSQL container, confirmed it loaded ten seeded patients (two VIP), and confirmed the two roles the `verify-rar` plugin needs are in place: `vva_admin` (the plugin's admin user) and `healthcare_read_template` (the inheritance parent for ephemeral roles).

What you'll do next

Move on to Start the MCP server to configure and start the only process that handles the on-behalf-of token, talks to Vault, and runs the actual SQL.

8. Start the MCP server

MCP Deployment

The MCP server is the only process in the stack that handles the on-behalf-of (OBO) token. The clinician's access token arrives from the agent as a Bearer header. The MCP server exchanges that token at IBM Verify (RFC 8693), gets back a scoped OBO containing the Rich Authorization Request the policy approved, hands the OBO to Vault, gets back an ephemeral PostgreSQL role with a 5-minute lease, runs a single SELECT, returns the row to the agent, and revokes the lease. Everything Verify-side, everything Vault-side, every database statement happens in this one process. Read its log if anything in the chain looks wrong.

Configure

Move into the MCP server directory and copy the example file:

```
cd mcp-server
cp .env.example .env
```

Open `.env` and fill in each value. The defaults that ship in the example file are correct for the local stack; the only ones you must change are the tenant, Token Exchange client id, and Vault token.

```
# From your IBM Verify tenant
VERIFY_TENANT_HOST=<your-tenant>.verify.ibm.com

# From infra/verify/verify-output.json (the bootstrap output)
VERIFY_TE_CLIENT_ID=<token-exchange-client-id>

# From the Vault bootstrap printout in the previous chapter
VAULT_TOKEN=hvs.<your-mcp-token>

# Defaults that match the local stack; change only if you bind something elsewhere
VAULT_ADDR=http://127.0.0.1:8200
VAULT_ROLE=healthcare-records
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
POSTGRES_DB=healthcare
PORT=3012
```

Notice there is **no** `VERIFY_TE_CLIENT_SECRET`. The MCP server fetches that secret from Vault on the first Token Exchange call (cached in process for five minutes), at `secret/data/VERIFY_TE_CLIENT_SECRET`. The Verify bootstrap (chapter 4) wrote it there automatically when you ran with `VAULT_ADDR + VAULT_TOKEN` set. The MCP's Vault token policy grants read on that path; nothing extra to configure here.

If your environment does not have Vault available and you ran the Verify bootstrap without `VAULT_ADDR + VAULT_TOKEN` set, the bootstrap printed the secret to your terminal instead. In that case uncomment the `VERIFY_TE_CLIENT_SECRET=` line in `.env.example` and paste the value. The MCP server uses the env var as a fallback when Vault is not reachable, so this same code runs against either pattern.

Be deliberate about each value. A wrong `VERIFY_TENANT_HOST` makes the Token Exchange fail with a DNS error; a stale `VAULT_TOKEN` makes Vault return 403 (both for the verify-rar call and for the secret fetch); a wrong `VAULT_ROLE` returns a `role not found` error. The test chapter has a one-paragraph diagnostic for each.

For the validated local demo, use `POSTGRES_PORT=15432` unless you intentionally changed the Docker compose port mapping. Vault uses the Docker-internal hostname `postgres:5432` when it creates ephemeral users; the MCP server, running on the host, uses `127.0.0.1:15432` to log in as those ephemeral users.

Install and run

<i>Screenshot placeholder: MCP server terminal showing listening on http://127.0.0.1:3012.</i>	
<pre>npm install npm run dev</pre>	
Expected:	
<pre>[vva-mcp-server] listening on http://127.0.0.1:3012 [vva-mcp-server] /healthz, /tool, /mcp registered</pre>	

The server runs in the foreground. Leave the terminal open; you will tail this log when you exercise the smoke test. If you want it in the background, you can run `npm run dev &`, but a foreground tail is more useful for the first run.

Smoke

<i>Screenshot placeholder: curl /healthz output from the MCP server.</i>
--

A trivial GET confirms the server is up and that nothing in its startup path threw an error.

<pre>curl http://127.0.0.1:3012/healthz</pre>
Expected:
<pre>{"status":"ok","service":"vva-mcp-server"}</pre>

If you see `Connection refused`, the server is not running on port 3012; look at the terminal where you ran `npm run dev`. If you see `503 service unavailable`, the server started but a downstream check failed; the log has the reason.

Behind the scenes

Once a real tool call arrives over the `/mcp` or `/tool` endpoint, the server performs steps 2 through 6 of the seven-step flow from the architecture chapter. The compressed walkthrough is worth reading once so the smoke-test output is not a mystery.

Step 2: receive the call. A POST arrives with `Authorization: Bearer <clinician access token>` and a JSON body naming a tool and arguments. The server validates the bearer is present (returns 401 if missing, before any other work) and looks up the tool handler.

Step 3: build the Rich Authorization Request, exchange the token. The tool handler builds an `authorization_details` block describing exactly the action it is about to do (`operation = read_patient_record`, target patient MRN, etc.). It POSTs an RFC 8693 Token Exchange to IBM Verify with the clinician's token as `subject_token`, the Token Exchange client credentials as the requesting client, and the RAR in the request body. For VIP reads, Verify returns `scope: mfa_challenge` and a challenge token;

the server then triggers a push to the clinician's phone (`/v1.0/authenticators/{id}/verifications`) and polls for the assertion. Once the user approves, the server completes the exchange with a `jwt_bearer` grant that re-sends the RAR. The result is an OBO JWT.

Step 4: present the OBO to Vault. The server POSTs to `verify-rar/creds/<VAULT_ROLE>` with the OBO JWT as the `X-Vault-Token` header. Vault's `verify-rar` plugin validates the JWT signature against the Verify JWKS, matches the embedded `authorization_details` to a `rar_mapping` on the role, and runs the role's SQL template against `vva_admin` to mint a fresh ephemeral PostgreSQL role with a 5-minute lease.

Step 5: query PostgreSQL. The server opens a connection using the ephemeral role's username and password, runs the single `SELECT` for the patient record, and closes the connection.

Step 6: revoke the lease, return. The server POSTs to `sys/leases/revoke` with the lease id, which drops the ephemeral PostgreSQL role immediately rather than waiting for the 5-minute TTL. It returns the patient record JSON to the agent.

Every one of those sub-steps is logged. If the smoke test fails, the line in the MCP server log that does not have a matching success pair tells you where in the chain the problem is.

What you just did

You configured the MCP server with the credentials from the IBM Verify bootstrap and the HashiCorp Vault bootstrap, started it on `127.0.0.1:3012`, and confirmed `/healthz` returns OK. The server is now ready to receive tool calls from an agent.

What you'll do next

Move on to Run the agent to install the Python + Strands agent's dependencies, configure your `.env`, and start the agent locally with `uvicorn`. The agent runs as a plain Python process — no AWS account, no managed runtime.

9. Run the agent

Run the agent

The agent is a small Python + Strands process that runs as a FastAPI server. It receives a clinician's prompt over HTTP, forwards the clinician's IBM Verify access token to the MCP server as an Authorization: Bearer header, lets Strands decide which MCP tool to call, and streams the model's text response back over Server-Sent Events. The agent has no security responsibilities of its own; everything in the security chain (Token Exchange, RAR, step-up MFA, OBO mint, the Vault verify-rar plugin, the ephemeral PostgreSQL credential) happens inside the MCP server you started in the previous chapter.

The agent has no AWS dependency. The cookbook's default LLM is Anthropic direct (claude-sonnet-4-5-20250929). The full list of swappable LLMs is in the next chapter.

Install Python deps

Screenshot placeholder: pip install -e . completed successfully inside the agent virtualenv.

Move into the agent/ directory and create a virtual environment. A venv keeps the agent's dependencies separate from anything else on your system.

```
cd agent
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e .
```

Expected: Successfully installed fastapi-... strands-agents-... mcp-... uvicorn-... plus a few transitive dependencies. The base install brings strands-agents[anthropic] which handles the default LLM_PROVIDER=anthropic path; the other providers (bedrock, openai, gemini) are pip extras and live in the next chapter.

If pip install -e . fails on a missing extra you have not configured, leave LLM_PROVIDER at its default of anthropic and try again.

Configure .env

Still inside agent/:

```
cp .env.example .env
```

Open .env and fill in the one value that has no default:

```
# Anthropic direct, from console.anthropic.com -> API keys
ANTHROPIC_API_KEY=sk-ant-...
```

The other defaults match the local stack the previous chapters brought up:

```
HEALTHCARE_MCP_URL=http://127.0.0.1:3012/mcp
LLM_PROVIDER=anthropic
```

If you want to point at a different MCP server (different port, different host, deployed somewhere), change `HEALTHCARE_MCP_URL`. If you want to use a non-default LLM, see Swapping the LLM.

Run locally

Screenshot placeholder: Uvicorn startup output for the healthcare agent on port 8080.

```
uvicorn healthcare_agent.main:app --host 127.0.0.1 --port 8080 --reload
```

Expected:

```
INFO: Will watch for changes in these directories: ['../agent']
INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO: Started reloader process
INFO: Started server process
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Leave that terminal open. You will hit it from the smoke-test chapter in another terminal. A `--reload` argument means the server picks up code edits automatically; remove it for a quieter run.

A `GET /healthz` confirms the server is up. From a second terminal:

```
curl -s http://127.0.0.1:8080/healthz
```

Expected:

```
{"status": "ok", "service": "healthcare-agent"}
```

The agent's two endpoints

The agent only has two routes. Both are intentional. Keep them small so the security-relevant behavior is obvious.

`GET /healthz` returns `{"status": "ok"}`. No authentication. It is for liveness checks.

`POST /invoke` is the actual agent turn. It takes:

- Authorization: Bearer `<clinician-access-token>` — the IBM Verify access token the clinician got from PKCE. The agent does not validate this token; it forwards it to the MCP server, which uses it as the RFC 8693 `subject_token`. If the token is missing, the agent returns `401 missing_bearer` immediately without doing any work.
- Content-Type: `application/json`
- A JSON body `{"prompt": "<the user's natural-language prompt>"}`

The response is Content-Type: `text/event-stream`. The agent yields the model's text output as SSE data: lines as Strands produces them, ending with `data: [DONE]`. The whole exchange typically takes 5-15 seconds for a non-VIP read (a few model turns to choose the tool, call it, and summarize the result), and 30 seconds or longer for a VIP read (because the MCP server is waiting for a clinician to approve a push on their phone, and the agent is just sitting on that long-running tool call). The smoke-test chapter exercises both.

Why FastAPI and Strands

A note on framework choice for engineers reviewing the code. Two pieces.

FastAPI is the HTTP server. It is one of the most-used Python web frameworks (alongside Flask and Django), it speaks ASGI, and it ships first-class Server-Sent Events support via its `StreamingResponse` class. The `healthcare_agent/main.py` file is around 50 lines top-to-bottom and you can read it end-to-end in a minute.

Strands is the agent loop. It is the model-agnostic agent framework (Anthropic, OpenAI, Gemini, Bedrock- Anthropic all behind one interface) maintained by AWS, but the framework itself has no AWS dependency. It does three things you would otherwise write by hand: the tool-use while-loop (model says `tool_use`, you call the tool, you post the result back), the MCP-tool-to-Strands-tool adaptation (so the MCP server's tool definitions become Strands tool definitions automatically), and the streaming event model. About 100 lines of agent loop logic that you do not have to maintain.

If you want to drop Strands and roll your own loop against the Anthropic SDK directly, the same `/invoke` endpoint shape and the same MCP wire protocol still work. Strands is the cookbook's choice for readability, not a hard requirement.

Hosting

This chapter ran the agent on your laptop. That is plenty for the cookbook's purpose: prove the chain works end-to-end with a real clinician token. When you are ready to put the agent somewhere your calling app can reach over the network, any host that runs Python and exposes an HTTP port will do. Three sensible defaults:

- **Container, on anything.** Build a tiny Dockerfile (FROM `python:3.11-slim`, `pip install -e .`, CMD `["uvicorn", "healthcare_agent.main:app", "--host", "0.0.0.0", "--port", "8080"]`) and ship it to your container platform of choice (a customer's EKS / GKE / OpenShift / Fly / Render).
- **A small VM.** `apt install python3.11 python3.11-venv`, clone the repo, run uvicorn under systemd. The cookbook's resource footprint is tiny.
- **A laptop, for evaluation.** What you are doing right now.

There is no "preferred" deployment in this cookbook because the security model does not depend on it. The same Verify Token Exchange + Vault verify-rar chain runs regardless of where the agent process lives. The hosting decision is your customer's.

What you just did

You installed the agent's Python dependencies in a virtual environment, configured the LLM provider (Anthropic direct by default), and started the agent on `127.0.0.1:8080`. The agent is ready to forward the next clinician token to the MCP server.

What you'll do next

Move on to End-to-end smoke test to mint a clinician token via OIDC + PKCE, hit `/invoke` with a real prompt, and watch the chain (Verify Token Exchange → Vault verify-rar → ephemeral PostgreSQL →

SELECT → return) execute end-to-end. The VIP step-up test in that chapter fires an MFA push to the clinician's phone before returning a VIP chart.

Full end to end test

Pre-requisites

- Docker stack up: `docker compose -f infra/docker-compose.yml ps` shows both `vva-postgres` and `vva-vault` as running.
- MCP server running on `127.0.0.1:3012`: `curl -s http://127.0.0.1:3012/healthz` returns the OK JSON.
- Agent running on `127.0.0.1:8080`: `uvicorn healthcare_agent.main:app --host 127.0.0.1 --port 8080` in its own terminal. `curl -s http://127.0.0.1:8080/healthz` returns the OK JSON.
- A clinician access token. The next section walks you through getting one.

How to get a clinician access token

```

rgraham@mac verify-vault-agentcore-reference % bash scripts/get-clinician-token.sh
source .clinician-token.env
echo "Token length: ${#CLINICIAN_TOKEN}"

1. Open this URL in your browser and sign in as your clinician:

https://myco-consulting.tryverify.ibm.com/oauth2/authorize?response_type=code&client_id=9edf2013-da8b-4317-be11-a80c65c931e8&redirect_uri=http://localhost:8765/cb&scope=openid+profile+email&code_challenge=FCOZYfVF2ACvtdq2Kywguqu-21w6_n-JV0mf2SF-R4&code_challenge_method=S256&state=eacf2b9aa188565

2. Waiting for the redirect on http://localhost:8765/cb ...

Got authorization code: zYxm7APH0Vpsq44jq34m...

3. Exchanging code for access token...

Success. Two ways to use the token:

1. Source the file written to /Users/rgraham/github-repos/verify-vault-agentcore-reference/.clinician-token.env:

source /Users/rgraham/github-repos/verify-vault-agentcore-reference/.clinician-token.env

2. Or copy this line into your shell:

export CLINICIAN_TOKEN=eyJhbGciOiJSUzI1NiIsImtpZCI6ImNlcmlzaSI6InR5cCI6ImF0K2p3dCJ9.eyJhcHBfaWQiOiI0NjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ4LWFmNWQtYmM0Nzc3MTU4MmJmIiwiaWF0IjE5MjA5Mzg0MjE3MTMzMjM3Dk1IiwiaXVkiOiJpbjllZGYyMDEzLWRRhOGItNDMxNy1iZTZEWE4MGW2NWm5MzF1OCj4dCjBjbGllbnRfawQioiI5ZWRRmMjAxMy1kYThiLTQzMtctYmUxMS1hODBiNjVjOTMxZTgiLCJleHAiOiE3NzkyMjY2NjksImdyYW50X2lkIjoieYzFhY2U5NTA2MTI0Yi00NGQ
```

The MCP server expects a real IBM Verify access token for a clinician in your tenant. The repo ships a one-shot helper at `scripts/get-clinician-token.sh` that runs the OIDC authorization-code + PKCE flow against the UI application your Verify bootstrap created. It handles the three traps that bite a hand-rolled PKCE script (base64url alphabet preserving case, state length ≥ 8 characters, the confidential-client `client_secret` Verify requires alongside PKCE) and writes the resulting token to a sourceable file so a long JWT never has to survive a terminal copy-paste.

Clinician token freshness matters too. The PKCE helper writes `.clinician-token.env` for the current session, but that access token is short-lived. If you see CSIAQ0158E with `authorization_grant` of type `user_code` does not exist or is invalid, or if you have been debugging long enough for the token to age out, run `scripts/get-clinician-token.sh` again, source the new `.clinician-token.env`, and retry the smoke test.

```
bash scripts/get-clinician-token.sh
```

The script prints an authorize URL, opens a one-shot HTTP listener on `localhost:8765` to catch the redirect (so the browser does not show a connection refused page), exchanges the code for an access token, and writes it to `.clinician-token.env` at the repo root (`chmod 600, gitignored`). It pulls the tenant host, UI client id, and UI client secret it needs from `infra/verify/verify-output.json` and from Vault KV — nothing for you to paste.

Expected (abridged):

1. Open this URL in your browser and sign in as your clinician:

`https://<your-tenant>.verify.ibm.com/oauth2/authorize?...`

2. Waiting for the redirect on `http://localhost:8765/cb ...`

Got authorization code: ...

3. Exchanging code for access token...

Success. Two ways to use the token:

1. Source the file written to `<repo>/clinician-token.env`:

```
source .clinician-token.env
```

Open the URL in a browser, sign in as the clinician whose `upn` matches the `CLINICIAN_UPN` value in `agent/.env`, and the script does the rest. Then load the token into your shell:

```
source .clinician-token.env
echo "${CLINICIAN_TOKEN:0:20}..."
```

The token is valid for ~1 hour (IBM Verify's default access-token lifetime). If a later step returns CSIAQ0158E ("authorization_grant of type user_code does not exist or is invalid" — misleading message; really means the token expired or was revoked), re-run `bash scripts/get-clinician-token.sh` to mint a fresh one. See troubleshooting #14 for the full diagnosis.

Run the test

```
Test 1: GET /healthz
{"status":"ok","service":"vva-mcp-server"}

Test 2: list_patients_for_clinician (no MFA expected)
{
  "patients": [
    {
      "mrn": "A0001",
      "display_name": "Alex Anderson",
      "dob": "1972-04-11",
      "primary_diagnosis": "Atrial fibrillation",
      "vip_flag": false
    },
    {
      "mrn": "A0002",
      "display_name": "Beth Bell",
      "dob": "1980-09-23",
      "primary_diagnosis": "Type 2 diabetes",
      "vip_flag": false
    }
  ]
}
```

Smoke test output, part 1.


```

    {
      "mrn": "A0099",
      "display_name": "CEO Taylor Thornton",
      "dob": "1969-10-08",
      "primary_diagnosis": "Major depression",
      "vip_flag": true
    }
  ]
}

Test 3: get_patient_record for A0001 (non-VIP, no MFA expected)

{
  "patient": {
    "mrn": "A0001",
    "display_name": "Alex Anderson",
    "dob": "1972-04-11",
    "primary_diagnosis": "Atrial fibrillation",
    "vip_flag": false,
    "primary_clinician": "ccarter@acmehealth.example"
  },
  "vipPath": false
}

Test 4: get_patient_record for A0042 (VIP; expect a push to yo

A push notification should land on the clinician's enrolled de
Approve it within 90 seconds.

{
  "patient": {
    "mrn": "A0042",
    "display_name": "Senator Riley Reed",
    "dob": "1958-05-20",
    "primary_diagnosis": "Coronary stent",
    "vip_flag": true,
    "primary_clinician": "ccarter@acmehealth.example"
  },
  "vipPath": true
}

All four tests returned. If Test 4 came back with the patient
the full chain (Verify Token Exchange → step-up MFA → OBO →
Vault verify-rar → ephemeral Postgres role → SELECT) is working
rgraham@mac verify-vault-agentcore-reference % █

```

Smoke test output, part 2.

The script `scripts/smoke-test.sh` runs four tests in order and prints results to stdout.

Validated update: the smoke script should print the HTTP status and response body on failures. That is much more useful than piping an empty or non-JSON response into `python -m json.tool` and seeing `Expecting value`. A failing tool call should now show HTTP 400/401/403/500 plus the JSON error body returned by the MCP server.

```
bash scripts/smoke-test.sh
```

Expected (four sections, abridged):

```

Test 1: GET /healthz
{"status":"ok","service":"vva-mcp-server"}

Test 2: list_patients_for_clinician (no MFA expected)
{
  "patients": [
    {"mrn": "A0001", "display_name": "Anderson, ...", ...},
    ... ten rows ...
  ]
}

Test 3: get_patient_record for A0001 (non-VIP, no MFA expected)
{
  "mrn": "A0001",
  "display_name": "Anderson, ...",
  "visits": [ ... ]
}

Test 4: get_patient_record for A0042 (VIP, expect a push to your phone)
  A push notification should land on the clinician's enrolled device.
  Approve it within 90 seconds.
  ... waits ...
{
  "mrn": "A0042",
  "display_name": "Thornton, ...",
  "visits": [ ... ]
}

All four tests returned. The full chain is working.

```

What to look for on Test 4

Test 4 is the proof. The flow is, in order:

38. The script POSTs `get_patient_record` for MRN A0042 to the MCP server.
39. The MCP server starts the Token Exchange against IBM Verify. Verify's access policy fires `ACTION_MFA_ALWAYS` because the RAR matches the `healthcareRarPresent` attribute.
40. IBM Verify pushes a notification to the clinician's enrolled mobile device.
41. The MCP server log shows the poll loop: `[verify] polling MFA assertion (attempt 1)... (attempt 2)....`
42. You tap **Approve** on your phone.
43. The MCP server log shows `[verify] MFA approved after N attempts`.
44. The MCP server hands the resulting OBO JWT to Vault. The PostgreSQL log shows the `CREATE ROLE, GRANT, SELECT, REASSIGN, and DROP ROLE` statements firing in under a second.
45. The MCP server returns the patient record JSON to the smoke-test script.

If your phone never rings, you did not get to step 3. The most common reasons are listed in the next section.

If it didn't work

401 on Test 1. The MCP server is not running, or it is running on a different port than 3012. Check the terminal running `npm run dev` and look for the `listening on http://127.0.0.1:3012` line. If the terminal

shows a different port, edit `mcp-server/.env` and set `PORT=3012` (or update `MCP=` at the top of the smoke-test script).

401 on Test 2. The clinician access token is expired or for the wrong tenant. IBM Verify access tokens default to a one-hour lifetime; if more than an hour has passed since you ran the authorize flow, repeat it. If the token is fresh, the most likely cause is `VERIFY_TENANT_HOST` in `mcp-server/.env` not matching the tenant that minted the token; double-check both.

Test 4 hangs. Two common causes. First, the access policy in IBM Verify is in state `IDLE` rather than `ACTIVE`. Sign into the Admin UI, open **Security** then **Policies**, find **HealthcareStepUp**, click into it, click Save then Publish. The policy state column should now read **Active**. Second, the clinician has no enrolled push factor. Sign into the IBM Verify self-service portal with that clinician's credentials and enroll the IBM Verify mobile authenticator.

Test 4 returns ``mfa_denied``. Either the user tapped **Deny** on the push, or the push timed out. The MCP server log shows which: `[verify] MFA denied by user` versus `[verify] MFA timeout after 120s`. Re-run the test and approve within 90 seconds.

VIP step-up smoke test

The cookbook's VIP patients (A0042 Senator Reed and A0099 CEO Thornton in the seed) trigger the Verify access policy's step-up MFA rule. To exercise the full chain end-to-end:

Prerequisite. The clinician account you signed in as must have an IBM Verify mobile-app push factor enrolled. Without one, the read returns `mfa_no_factor` (see troubleshooting #8).

```
curl -N -X POST http://127.0.0.1:8080/invoke \
  -H "Authorization: Bearer $CLINICIAN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Read patient A0042"}'
```

Expected sequence:

46. The agent's reply pauses (the MCP is waiting on Verify's MFA challenge).
47. A push notification lands on the clinician's phone: "Approve healthcare action".
48. Approve the push within 120 seconds.
49. The agent returns Senator Reed's chart with `mrn`, `display_name`, `dob`, `primary_diagnosis`, `primary_clinician`, and `vip_flag: true`.

What just happened end-to-end:

- The agent called `get_patient_record` with `mrn=A0042`.
- The MCP server's Step A discovery mint ran under the `patient_read` RAR (no MFA — policy `ACTION_ALLOW`). The plugin returned an ephemeral Postgres role, the row was read, `vip_flag` came back true.
- The MCP server's Step B re-fired Token Exchange with the `patient_read_vip` RAR. Verify's CELX matched, the policy fired `ACTION_MFA_ALWAYS`, and `/oauth2/token` returned scope: `mfa_challenge` with a challenge access token.
- The MCP server used the challenge token to call `/v2.0/factors`, found the clinician's userPresence factor, posted a verification to `/v1.0/authenticators/{id}/verifications`, got back a `transactionUri`, and polled until state: `VERIFY_SUCCESS` arrived with an assertion JWT.

- The MCP server re-called `/oauth2/token` with `grant_type=jwt-bearer`, the assertion, and the SAME RAR re-attached (Verify does not propagate `authorization_details` through the second leg by default).
- Verify returned a post-MFA OBO carrying `authorization_details` signed into the JWT.
- The MCP server presented that OBO to the `verify-rar` Vault plugin, which validated the RAR shape against the role's `rar_mappings`, minted a fresh 5-minute Postgres role, the MCP server ran the full `SELECT`, and the chart returned.

Try the denial paths too:

```
# Deny the push when it arrives. Agent surfaces an mfa_denied error.
curl -N -X POST http://127.0.0.1:8080/invoke \
  -H "Authorization: Bearer $CLINICIAN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Read patient A0099"}'

# Ignore the push entirely; wait 2+ minutes. Agent surfaces an mfa_timeout error.
curl -N -X POST http://127.0.0.1:8080/invoke \
  -H "Authorization: Bearer $CLINICIAN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Read patient A0042"}'
```

Both error paths are documented in troubleshooting #9 and #10.

What you just did

You generated a clinician access token, ran four test calls against the MCP server, and exercised both the no-MFA read path and the step-up MFA path. You watched a push notification land on your phone, approved it, and saw the corresponding patient record come back. The full identity chain works end-to-end on your laptop.

What you'll do next

If you want to try a non-default LLM provider (Bedrock, OpenAI, or Gemini), move on to [Swapping the LLM](#). If you want a code-level walkthrough of how the MCP server enforces the security chain, [Anatomy of an MCP Call](#) quotes the load-bearing files verbatim. If something went wrong above, jump to [Troubleshooting](#).

11. Swapping the LLM

LLM Options

The agent is built on the Strands framework, which abstracts the LLM provider behind a single Agent interface. The same agent code, the same tools, the same MCP server, the same IBM Verify and HashiCorp Vault chain, all work against four different model providers. This chapter is the table that says how to switch.

The four providers

The agent ships with adapters for Anthropic direct (the cookbook default), AWS Bedrock (Anthropic on Bedrock), OpenAI, and Google Gemini. Pick one based on what your organization is licensed for. None of them changes the identity or authorization chain; only the model that produces the next token changes.

Provider	LLM_PROVIDER value	Extra to install	Required env vars	Default model id
Anthropic direct (default)	anthropic	none (default)	ANTHROPIC_API_KEY	claude-sonnet-4-5-20250929
AWS Bedrock (Anthropic)	bedrock	<code>pip install -e '[bedrock]'</code>	AWS_REGION, AWS credentials via the SDK chain	us.anthropic.claude-sonnet-4-5-20250929-v1:0
OpenAI	openai	<code>pip install -e '[openai]'</code>	OPENAI_API_KEY	gpt-4o
Google Gemini	gemini	<code>pip install -e '[gemini]'</code>	GEMINI_API_KEY	gemini-2.0-flash

Set LLM_PROVIDER in agent/.env. If you switch providers, also set the matching API key (or AWS credentials for Bedrock). Restart uvicorn for the change to take effect.

The agent does not care which one you pick

Everything below the model layer is provider-independent. The tools (`list_patients_for_clinician`, `get_patient_record`) are the same. The MCP server protocol is the same. The IBM Verify Token Exchange does not see what model the agent used. The Rich Authorization Request is identical regardless of provider. The HashiCorp Vault flow, the ephemeral PostgreSQL role, and the SELECT all happen the same way. Swap the model, the identity chain does not move.

Why Anthropic direct by default

Two reasons. First, the cookbook's customer story is "this is the identity pattern for an MCP server; pick your hosting" — and the lowest-friction LLM path that does not couple the cookbook to a cloud vendor is Anthropic direct. You generate an ANTHROPIC_API_KEY at console.anthropic.com, drop it in .env, and start uvicorn. No cloud account, no role chain, no Bedrock model-access form to submit.

Second, Strands' default Claude Sonnet 4.5 builds on Anthropic's strongest tool-use model. The OpenAI and Gemini adapters work but their tool-calling reliability is below Claude on this cookbook's scenarios; pick them when your organization is committed to that provider, not because they are technically superior here.

Why Bedrock if you have an AWS account

Bedrock is the AWS-native path. When your organization already has AWS credentials in place (IAM roles, KMS keys, billing), Bedrock model invocations bill through the same account and the credentials chain through the same SDK. The default model id (`us.anthropic.claude-sonnet-4-5-20250929-v1:0`) is the US cross-region inference profile, which AWS recommends over the global profile for production traffic.

Bedrock is the right call for customers who have already pushed their entire AI estate through Bedrock. It is not the right call for a fresh evaluation; Anthropic direct lets your customer judge the pattern without standing up an AWS account first.

Local models (no vendor)

Strands does not currently ship a first-class adapter for local LLM servers (Ollama, vLLM, llama.cpp). If you need to run fully air-gapped:

50. Run an OpenAI-compatible local LLM server (Ollama and vLLM both expose one).
51. Set `LLM_PROVIDER=openai` and point `OPENAI_API_BASE` at your local server (the OpenAI Python SDK respects this env var).
52. Use a model that is good enough at tool use; Llama 3.1 70B is the realistic ceiling for laptops with GPU access, and its tool-use quality is noticeably below Claude Sonnet.

The security chain stays identical — IBM Verify still authorizes, HashiCorp Vault still mints the ephemeral credential. Only the model changes. This path is supported but not tested by the cookbook's smoke test.

What you just did

You read the four-row table that says what to install and what env vars to set for each LLM provider, and you confirmed that the identity chain does not change when you swap the model.

What you'll do next

If you have not run End-to-end smoke test yet, do that. If you have, move on to Anatomy of an MCP Call to walk the code that implements the security chain, or Troubleshooting if you have hit a snag.

12. Anatomy of an MCP call

Anatomy of an MCP Call

The smoke test in chapter 10 proved that the chain works. This chapter explains the *code* that makes each step work. Three files inside `mcp-server/src/` carry the security model. The snippets below are quoted verbatim from the repo; the prose around each one names which property of the identity chain it enforces.

Read this chapter with `mcp-server/src/` open in a second window. The point is not to discover anything you have not already seen — the smoke test exercised every line of this code. The point is to *name* the load-bearing line in each file so a security reviewer can audit them in 15 minutes instead of an hour.

1. Reject unauthenticated requests before doing any work

The very first check in `mcp-server/src/index.ts`:

```
app.post('/tool', async (req, res) => {
  const bearer = (req.header('authorization') || '').replace(/^Bearer /, '');
  if (!bearer) return res.status(401).json({ error: 'missing_bearer' });
  // dispatch only happens with a bearer present
  const result = await dispatchTool(toolName, args, bearer);
  res.json(result);
});
```

A request without a `Authorization: Bearer` header never reaches a tool handler. Notice what the MCP server does *not* do here: it does not parse the bearer, it does not check the bearer against a local key, it does not validate the bearer's expiration. Validating the bearer happens at IBM Verify in step 3, where Verify is the authoritative source for whether the token is still active. The MCP server's job at this layer is just "presence check."

This is the first guarantee in the chain: no anonymous calls. A request that arrives with no Bearer header gets 401 `missing_bearer` in under a millisecond.

2. Every tool call builds a fresh, action-specific RAR

The relevant lines from `mcp-server/src/tools/get-patient-record.ts`:

```
// VIP patients trigger the policy's ACTION_MFA_ALWAYS rule.
// The RAR is what Verify's CELX rule matches against.
const vipCred = await mintCred({
  rarType: 'urn:smt:agent:healthcare',
  rarAction: 'patient_read_vip',
  patientMrn: mrn,
  claims,
  exchangeToken,
});
```

The MCP server does not decide whether step-up MFA fires. It *declares* the action it is about to perform (read a VIP patient's chart, by MRN) and lets Verify's access policy decide. The CELX rule in

infra/verify/bootstrap-verify.ts matches patient_read_vip and fires ACTION_MFA_ALWAYS; non-VIP reads use a different action verb (patient_read) and get ACTION_ALLOW with no push.

This is the second guarantee: declared intent, not asserted authority. The RAR is the MCP server's *request* for permission, not its claim that it already has permission. Verify is the only entity that can grant the request.

3. The Token Exchange handler implements the two-leg MFA flow verbatim

The non-obvious part. The relevant block from mcp-server/src/verify/token-exchange.ts:

```
// Leg 1: POST /oauth2/token with grant_type=token-exchange + RAR
// -> 200 OK with scope: "mfa_challenge" if policy demands MFA
// -> 200 OK with real OBO if policy allows
// Leg 2 (only if mfa_challenge): drive the MFA push internally, then
// POST /oauth2/token with grant_type=jwt-bearer + the MFA assertion
// + the SAME RAR re-attached so the post-MFA OBO carries
// policy-attested authorization_details.
```

The same RAR object is sent on both legs. IBM Verify is the only place that ever signs authorization_details into a JWT; the MCP server's role is to construct the RAR and re-submit it on the second leg. Re-attaching the RAR on the jwt_bearer leg is non-obvious — Verify does not propagate authorization_details through automatically, and an early version of this code missed that — so the included token-exchange.test.ts has vitest coverage for the path. The full helper code is in the same file: triggerOAuthMfaPush, pollOAuthMfaStatus, and exchangeMfaAssertionWithRAR.

This is the third guarantee: policy-attested attribution. Whatever RAR appears in the resulting OBO JWT was approved by IBM Verify's policy engine, not asserted by the MCP server. Vault's verify-rar plugin validates the JWT signature before it matches the RAR; a forged RAR would not survive the signature check.

4. The MCP server presents the OBO directly to Vault

After the OBO arrives, the MCP server passes it to Vault as the X-Vault-Token:

```
const res = await fetch(`${VAULT_ADDR}/v1/verify-rar/creds/${role}`, {
  method: 'POST',
  headers: { 'X-Vault-Token': obo, 'Content-Type': 'application/json' },
  body: JSON.stringify({ claims }),
});
```

The OBO is *its own* Vault credential. Vault's verify-rar plugin validates the OBO JWT against the IBM Verify JWKS, finds the embedded authorization_details, walks the role's rar_mappings, and matches. If the match succeeds, the plugin mints a fresh PostgreSQL role with a 5-minute lease and returns the role's username/password to the MCP server. If the match fails (RAR shape the role does not allow, expired JWT, bad signature), Vault returns 403.

This is what makes the entire chain "policy enforced at every hop" rather than "policy enforced once at the front door." Even if a sophisticated attacker convinced the MCP server to skip the Verify step, they would still need a valid Verify-signed OBO to get a database credential out of Vault.

5. The ephemeral credential runs one SELECT, then the lease is revoked

The lifetime of the credential is one statement:

```
const pool = new Pool({ user: cred.username, password: cred.password, ... });
try {
  const result = await pool.query(sql, args);
  return result.rows;
} finally {
  await pool.end();
  await revokeLease(cred.leaseId);
}
```

`revokeLease()` calls `POST /v1/sys/leases/revoke`, which immediately drops the ephemeral PostgreSQL role. Vault audit records both the mint and the revoke, joined by the lease id. PostgreSQL records the SQL statement under the ephemeral role's name. A SIEM joining on the OBO `jti` reassembles the chain end-to-end.

This is the final guarantee: no credential outlives the operation it was minted for. The 5-minute TTL is a backstop; the explicit revoke is the primary mechanism.

The "almost nothing" list, restated as code

These four properties are the entire local security boundary of the MCP server:

- 53. The bearer-presence gate is one line: `if (!bearer) return res.status(401).json(...)`.
- 54. The per-call RAR construction lives inside each tool handler; no shared RAR cache, no RAR memoization.
- 55. The Token Exchange handler is one file (`verify/token-exchange.ts`); it is the only place in the codebase that POSTs to `/oauth2/token`.
- 56. The Postgres call uses Vault's ephemeral credential and revokes it in a `finally`; no long-lived pool, no admin connection string in `.env`.

If a reviewer asks "show me where the authorization logic lives in this codebase," the honest answer is: it does not. The authorization decision is in IBM Verify. The credential-minting decision is in HashiCorp Vault. The MCP server is the orchestrator that calls both at the right time.

What you just did

You read the five load-bearing pieces of code in `mcp-server/src/`. Together, they implement the identity chain from chapter 2. You can now point at a specific file and line for each of the three guarantees Verify provides and the four properties the MCP server is forbidden from violating.

What you'll do next

If you have not run End-to-end smoke test yet, run it now and watch the code in this chapter execute in real time. If something in the chain misbehaves during your own testing, Troubleshooting covers the most

common gotchas. Otherwise, move on to Logging for an enterprise SIEM to see how the audit surfaces in this stack join on a single jti.

13. Troubleshooting

Troubleshooting

The entries below are ordered roughly by the order in which you encounter them during a fresh setup. Each one has the symptom you actually see, the diagnosis (what is really happening), and the fix.

Table of contents

- 57. `pip install -e .` fails with error: Microsoft Visual C++ ... on Windows
- 58. `uvicorn` command not found after `pip install -e .`
- 59. Agent returns 401 `missing_bearer`
- 60. Agent returns 500 with `mcp.client.streamable_http: ConnectError`
- 61. HashiCorp Vault returns 403 on `verify-rar/creds/<role>`
- 62. IBM Verify Token Exchange returns `scope: mfa_challenge` instead of an OBO
- 63. Token Exchange returns CSIAQ0158E "authorization_grant of type user_code does not exist or is invalid"
- 64. VIP read fails with `mfa_no_factor`
- 65. VIP read fails with `mfa_denied`
- 66. VIP read fails with `mfa_timeout`

• **

1. `pip install -e .` fails with C-compiler required

Symptom. Running `pip install -e .` in the agent's virtual environment fails partway through with an error mentioning a missing C compiler (typical on Windows: error: Microsoft Visual C++ 14.0 or greater is required).

Diagnosis. A transitive dependency in the agent's chain (usually `httpx`'s native `h11` or one of Strands' SDK extras) needs to build a small C extension if no pre-built wheel matches your Python version and platform.

Fix. Easiest path is to install Python 3.11 or 3.12 (not 3.13+) — these have full wheel coverage for everything in this dependency tree on Mac, Linux, and Windows. If you must use an unreleased Python, install the build toolchain (`apt install build-essential` on Linux, Xcode CLI tools on Mac, the Visual Studio Build Tools on Windows) and retry.

• **

2. `uvicorn` command not found after `pip install -e .`

Symptom. `pip install -e .` reports success, but `uvicorn healthcare_agent.main:app ...` returns command not found (or `uvicorn: not found`).

Diagnosis. Either you forgot to source `.venv/bin/activate` (the `venv`'s `bin/` directory is not on your `PATH`) or you ran `pip install` in a `venv` but `uvicorn` from your shell.

Fix.

```
cd agent
source .venv/bin/activate
which uvicorn # should print ../agent/.venv/bin/uvicorn
uvicorn healthcare_agent.main:app --host 127.0.0.1 --port 8080
```

If which uvicorn still does not return the venv path after activation, the install failed silently — re-run `pip install -e .` and check for errors.

- **

3. Agent returns 401 missing_bearer

Symptom. POST `/invoke` returns:

```
{"detail": "missing_bearer"}
```

Diagnosis. The request did not carry a `Authorization: Bearer <token>` header, or the header was empty after stripping the `Bearer` prefix. The agent rejects unauthenticated requests at the front door — it does not look up tools or call the MCP server until the bearer is present.

Fix. Pass the clinician token (mint it with `bash scripts/get-clinician-token.sh && source .clinician-token.env`):

```
curl -N -X POST http://127.0.0.1:8080/invoke \
-H "Authorization: Bearer ${CLINICIAN_TOKEN}" \
-H "Content-Type: application/json" \
-d '{"prompt": "List my patients"}'
```

- **

4. Agent returns 500 with `mcp.client.streamable_http: ConnectError`

Symptom. The agent log shows a Python traceback ending in `httpx.ConnectError: All connection attempts failed against 127.0.0.1:3012`.

Diagnosis. The agent is reaching for the MCP server at `HEALTHCARE_MCP_URL` (default `http://127.0.0.1:3012/mcp`), but nothing is listening there. Either the MCP server is not running, it is running on a different port, or your `agent/.env` points at the wrong URL.

Fix. In a second terminal:

```
curl -s http://127.0.0.1:3012/healthz
```

Should return `{"status": "ok", "service": "vva-mcp-server"}`. If you get connection refused, start the MCP server (`cd mcp-server && npm run dev`). If the MCP server is on a non-default port, update `HEALTHCARE_MCP_URL` in `agent/.env` and restart uvicorn.

- **

5. HashiCorp Vault returns 403 on verify-rar/creds/<role>

Symptom. The MCP server logs Vault: permission denied when it POSTs to verify-rar/creds/healthcare-records, even though the VAULT_TOKEN you copied from the bootstrap looks fine. The Vault CLI command `vault read verify-rar/creds/healthcare-records` works without complaint when run as the same token.

Diagnosis. This is the subtle one. The verify-rar plugin path is registered with HashiCorp Vault for both `ReadOperation` and `UpdateOperation`. Vault's HTTP layer does not deserialize a JSON request body on a GET, so the plugin's read handler cannot accept the OBO. Workloads have to POST, and POST corresponds to the update capability in Vault, not read. If your policy grants only read on this path, the CLI test passes (it uses the read path) but the workload POST fails with 403 (it needs update).

Fix. The included `healthcare-mcp.hcl` policy grants both capabilities, so the cookbook works out of the box. If you ever write your own policy for a similar plugin, grant both:

```
path "verify-rar/creds/healthcare-records" {
  capabilities = ["read", "update"]
}
```

• * *

6. IBM Verify Token Exchange returns scope: mfa_challenge instead of an OBO

Symptom. The MCP server log shows Token Exchange returned `scope=mfa_challenge`. The clinician's phone rings but the MCP server appears to do nothing with the resulting assertion.

Diagnosis. This is not an error. This is the normal IBM Verify pattern for an access policy that fires `ACTION_MFA_ALWAYS`. The first leg of the exchange returns a 200 OK with a `mfa_challenge` token (not a real OBO) and a transaction URI for polling. The MCP server is expected to:

67. Use the challenge token to call `/v2.0/factors` for the user.
68. Trigger a push at `/v1.0/authenticators/{factor_id}/verifications`.
69. Poll the transaction URI until the user approves.
70. Submit a second exchange using `grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer` with the assertion JWT as the assertion parameter, re-sending the original `authorization_details`.

The result of that second leg is a real OBO with the RAR baked in.

Fix. The included `mcp-server/src/verify/token-exchange.ts` already implements this two-leg flow. If you see the `mfa_challenge` log line, the next several lines should show polling MFA assertion (attempt 1)... (attempt 2)... approved. If you do not see those, the polling helper is not being invoked; the most likely cause is a mismatched code copy from an earlier reference that lacked the helper. Pull from the cookbook's main branch.

A common related symptom: if you forget to re-send `authorization_details` on the second-leg `jwt_bearer` call, the OBO comes back without a RAR and HashiCorp Vault returns `no_rar_mapping_match` on the next step. The cookbook handles this; flag it if you ever fork the code.

- **

7. Token Exchange returns CSIAQ0158E

"authorization_grant of type user_code does not exist or is invalid"

Symptom. The MCP server log (or a direct curl to `/mcp` with a `tools/call`) returns:

```
Leg 1 token exchange failed: 400 {"error":"invalid_request","error_description":"CSIAQ0158E The authorization_grant of type user_code does not exist or is invalid."}
```

The agent's chat reply surfaces this as a generic "authentication issue" or "authorization problem".

Diagnosis. The error message mentions `user_code` but that is misleading. CSIAQ0158E is IBM Verify's general "the subject_token presented to Token Exchange is no longer valid" error. The user's access token has either:

- Expired (default Verify access token TTL is 1 hour),
- Been explicitly revoked by Verify (Continuous Access Evaluation, manual revocation, or session-end), or
- Was minted for a different OIDC application than the one the Token Exchange app trusts.

The most common cause is "I got the token an hour ago and forgot to refresh."

Fix. Re-mint the clinician token and retry:

```
bash scripts/get-clinician-token.sh
source .clinician-token.env

# Retry your call
curl -N -X POST http://127.0.0.1:8080/invoke \
  -H "Authorization: Bearer $CLINICIAN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"prompt": "List my patients"}'
```

If you minted a brand-new token and still see CSIAQ0158E, the issue is configuration rather than expiry. Check that the Token Exchange app's `clientGroups.tokenExchange` (in the Verify Admin UI under the TE app's Sign-on tab) lists the UI app's `clientId` as a trusted subject. Verify rejects `subject_tokens` minted by clients not in that list with CSIAQ0158E. The cookbook bootstrap does not set this explicitly because in the default flow the same UI app issues the `subject_token` and is implicitly trusted; if you swap in a different UI app you may need to add it.

If the token is fresh and Verify still rejects the exchange, then debug tenant configuration: policy status, Token Exchange app entitlements, `clientGroups.tokenExchange`, and the TE client secret stored in Vault KV. If the exchange succeeds but the database read fails with `role does not exist`, check the Postgres host/port split: Vault must create the role in the Docker database, and the MCP server must connect to that same Docker database on the host-published port.

• **

8. VIP read fails with mfa_no_factor ("user has no registered userPresence factor")

Symptom. Asking the agent to read a VIP patient (e.g. A0042) returns an error message mentioning `mfa_no_factor`. The MCP server log shows `triggerOAuthMfaPush: user has no registered userPresence factor`.

Diagnosis. The clinician account you signed in as does not have an IBM Verify mobile-app push factor enrolled on the Verify tenant. The VIP step-up flow requires one; the policy demands MFA, the MCP server tries to trigger a push, and no enrolled push factor means there is nothing to push to.

Fix. Enroll the IBM Verify mobile app for the clinician account:

71. Install the IBM Verify app on your phone (iOS App Store / Google Play).
72. In the Verify Admin UI: Identity -> Users -> select your clinician account -> Authentication factors -> Add factor -> IBM Verify mobile.
73. Scan the resulting QR code with the IBM Verify app.
74. Re-run the VIP read prompt. The push should fire.

• **

9. VIP read fails with mfa_denied

Symptom. Push notification appears on the clinician's phone, the clinician taps Deny (or the verification is rejected), the agent's reply mentions `mfa_denied` or `User denied push`.

Diagnosis. This is the policy working as intended — the clinician chose not to authorize the action. No fix needed. The MCP returned 4xx-equivalent, the agent surfaced the denial, and Verify recorded the event in its audit log.

Fix. None required. If the clinician intended to approve, ask them to retry: re-run the prompt and approve the next push within the 120-second window.

• **

10. VIP read fails with mfa_timeout

Symptom. Push notification appears on the clinician's phone, the clinician does not approve or deny within ~120 seconds, the agent's reply mentions `mfa_timeout`.

Diagnosis. The cookbook's default MFA poll timeout is `MFA_POLL_TIMEOUT_MS=120000` (two minutes). If the clinician was away from their phone or the push notification was missed, the MCP server stops polling and returns the timeout.

Fix. Re-run the prompt and respond to the push promptly. If the customer's environment legitimately needs a longer window (slow networks, controlled environments where the user must walk to a secured device), raise the timeout in `mcp-server/.env`:

```
MFA_POLL_TIMEOUT_MS=300000 # 5 minutes
```

Restart the MCP server (`npm run dev`) for the new value to take effect.

- * *

What you just did

You read the ten common gotchas and you know how to diagnose each one from its first observed symptom. Most of these took someone an hour to discover the first time and ten minutes to fix once they knew where to look. Save yourself the hour.

What you'll do next

If you have not run End-to-end smoke test yet, do that. If you have, move on to Anatomy of an MCP Call to walk the code that implements the security chain, or Logging for an enterprise SIEM to see how the audit surfaces in this stack join on a single `jti`.

14. Logging for an enterprise SIEM

Auditing and Logging

Every tool call leaves audit records in five different places. Each is a standard log surface your SIEM already knows how to ingest, and all five entries for the same call share a single `jti` claim from the on-behalf-of token. A SOC analyst can pull one chain end-to-end from any of the five surfaces and replay every step. This chapter is the table that says which is which, and the section that explains the join.

The five log surfaces

Surface	What it captures	Forward to your SIEM via
Agent process log	Per-invocation request id, prompt, the tool calls Strands decided to make, duration, any errors. Streamed to stdout/stderr by uvicorn.	Tail to syslog and forward via your standard agent (Splunk Universal Forwarder, Sentinel Log Analytics agent, QRadar WinCollect, Chronicle Forwarder). Or wrap the process in <code>systemd-cat / logger</code> to land it in <code>journald</code> .
MCP server log	The OBO <code>jti</code> , the RAR shape that went to Vault, the lease id Vault returned, the duration of each step.	Same syslog forwarder pattern as the agent log.
HashiCorp Vault audit log	Every Vault operation: the OBO submission, the <code>cred_issued</code> event from the <code>verify-rar</code> plugin, the lease revoke. The plugin extracts the OBO <code>jti</code> into the audit record.	Vault's audit device writes to a JSON file or syslog; both are standard SIEM sources. Enable with <code>vault audit enable file file_path=/vault/audit.log</code> .
PostgreSQL session log	Every SQL statement, with the ephemeral role name (<code>v-healthcare-...</code>) and the connection start and end.	Postgres <code>log_destination = stderr</code> to file, then the same syslog forwarder. The MCP server sets <code>application_name</code> on the connection to include the OBO <code>jti</code> , so every PG log line for the call carries the join key.
IBM Verify event stream and CAEP	Every Verify policy evaluation, MFA challenge issued, MFA approved or denied, OBO issued. Each event carries <code>requestid (= grant_id)</code> and the user principal.	IBM Verify's CAEP transmitter pushes Security Event Tokens (SETs) to any subscriber that speaks OpenID SSF. Subscribe your SIEM as a receiver.

If you already aggregate Linux and database logs into a SIEM, you already ingest four of these five. The fifth (IBM Verify) speaks the standards your SOC is moving toward anyway (OpenID Shared Signals Framework and Continuous Access Evaluation Profile).

The jti is the join key

This is the headline of the chapter. The OBO JWT minted by IBM Verify carries a unique `jti` (JWT id) claim. That same `jti` appears in:

- 75. IBM Verify's event stream.** Verify emits a `verify:risk` event when the policy evaluates and an `agent:obo_issued` event when the OBO is minted. Both carry the OBO `jti` as a field.
- 76. The Vault audit entry for the credential mint.** The `verify-rar` plugin extracts the `jti` from the OBO it received and writes it into the audit record's `request.data.jti` field (and into the response's `data.jti`).
- 77. The `verify-rar` plugin's `'cred_issued'` event.** A separate higher-level event the plugin emits alongside the audit entry, joinable to the same `jti`.

78. **The Vault audit entry for the lease revoke.** The plugin attaches the original `jti` to the revoke metadata.
79. **The MCP server log.** Every log line in the tool call's flow carries the `jti` in its structured payload.
80. **The PostgreSQL session log.** The MCP server sets `application_name` on the connection string to include the `jti` prefix, so every PostgreSQL log line for that connection carries it.

One SIEM query joins on `jti` and reconstructs the full chain. Splunk: `index=* jti="<value>" | sort _time`. Sentinel KQL: `union * | where Properties contains "<jti>" | order by TimeGenerated`. Same for any other SIEM that does field extraction on JSON or syslog. No correlation engine is required because the correlation id is already in every record.

What the customer does NOT have to build

The conventional answer to "I want to audit an AI agent" is to build a separate telemetry pipeline: an agent-event aggregator, a custom correlation engine, a bespoke parser for whatever the agent framework emits. None of that is necessary here. The reasons:

- **No aggregator.** The five surfaces above already exist on systems your operations team manages. No new daemon to run.
- **No correlation engine.** The `jti` is the correlation id. Your SIEM's existing query engine is the correlation engine.
- **No bespoke parser.** Vault audit is JSON. PostgreSQL log is the standard `log_line_prefix` plus statement. CAEP SETs are standard JWS. Every SIEM ships parsers for all three.
- **No agent telemetry pipeline.** The agent itself does not need to emit custom telemetry. The audit trail is produced by the systems the agent talked to, not by the agent.

Standards-based, not proprietary

Every audit surface in this stack speaks an open standard. There is no proprietary protocol you have to learn or implement support for. The list:

- **RFC 8693** OAuth 2.0 Token Exchange (the protocol Verify and the MCP server speak for OBO).
- **RFC 9396** OAuth 2.0 Rich Authorization Requests (the structure of the `authorization_details` block that survives end-to-end).
- **OpenID Shared Signals Framework (SSF).** The IBM Verify event stream conforms to SSF.
- **OpenID Continuous Access Evaluation Profile (CAEP).** Security Event Token (SET) format the transmitter emits.
- **Vault audit JSON.** A documented HashiCorp schema.
- **PostgreSQL log_line_prefix.** Standard, configurable, and the same for every SIEM that has ever ingested PostgreSQL logs.
- **syslog / journald.** The two standard Linux log delivery mechanisms; every SIEM has native ingestion.

If your enterprise architecture review asks for "no proprietary audit pipelines," this stack passes by construction.

What you just did

You read the five log surfaces, you saw the join key that ties them together (`jti`), and you confirmed that every surface in the chain speaks an open standard your SIEM already ingests.

What's next

You have reached the end of the cookbook. The working chain you have on your laptop is the same chain you would run anywhere else, with the agent and MCP server running on whatever hosting your customer chooses (a container platform, a small VM, a developer laptop) and your existing SIEM in place of the local log tails. Nothing about the security model changes when you move from a laptop to production.

If anything in the chain is misbehaving, Troubleshooting covers the most common gotchas with diagnosis-first prose. If you want a code-level walkthrough of how the MCP server enforces the security chain, the Anatomy of an MCP Call chapter quotes the load-bearing files verbatim.